LPI · LINUX ESSENTIALS · 010-160 · V1.6

جلد اول

لینوکس مقدماتی

LINUX ESSENTIALS

جلد اول · از نصب تا تسلط بر خط فرمان

ساخت آزمایشگاه روی VMware، جامعه‌ی لینوکس و متن‌باز، مدیریت بسته، Bash، راهنما، پیمایش فایل‌سیستم، مدیریت فایل، Pipe، grep، find و آرشیو؛ همه با سوییچ‌های رایج، مثال واقعی و لابراتوار گام‌به‌گام.

```
student@ubuntu-lab: ~

student@ubuntu-lab:~$ whoami
student
student@ubuntu-lab:~$ ls -lah ~/lab
drwxr-xr-x 4 student student 4.0K docs
student@ubuntu-lab:~$ find /etc -name '*.conf' | wc -l
137
student@ubuntu-lab:~$ grep -rn 'Port' /etc/ssh/sshd_config
14:#Port 22
student@ubuntu-lab:~$
```

TOPIC 1
جامعه‌ی لینوکس و متن‌باز

TOPIC 2
پیدا کردن راه در سیستم

TOPIC 3
قدرت خط فرمان (بخش اول)

UBUNTU 24.04 LTS
VMware Workstation 17+
LPI Linux Essentials 1.6

مدرس و طراح لابراتوار

محمدرضا بیدختی

MOHAMMAD REZA BEIDOKHTI

درباره‌ی این جزوه

About this course book

این جزوه برای کلاس **لینوکس مقدماتی** و آمادگی آزمون بین‌المللی **LPI Linux Essentials** (کد `010-160` ، نسخه‌ی ۱٫۶) نوشته شده و در دو جلد منتشر می‌شود. مخاطب آن کسی است که تا امروز با لینوکس کار نکرده و شاید فقط آن را نصب کرده است. پس از صفر شروع می‌کنیم: سیستم‌عامل و کرنل چیست، لینوکس از کجا آمد، ترمینال چیست و چرا مدیران سیستم این‌قدر به آن علاقه دارند، چطور در درخت فایل‌ها راه برویم، فایل‌ها را بسازیم، کپی کنیم، پیدا کنیم و متن داخل آن‌ها را جستجو کنیم.

هدف این جزوه فقط «قبول‌شدن در آزمون» نیست. می‌خواهیم وقتی جزوه تمام شد، بتوانید روی یک سرور واقعی Ubuntu بنشینید، بفهمید چه خبر است و بدون ترس کار کنید. برای همین هر فرمان را با **ساختار کلی**، **جدول سوییچ‌های رایج**، **مثال‌های واقعی با خروجی** و **هشدارهای عملی** آورده‌ایم و بعد از هر درس یک **لابراتوار سناریومحور** داریم که روی ماشین مجازی خودتان اجرا می‌کنید.

این جزوه چطور چیده شده است؟

ترتیب مطالب بر اساس پنج سرفصل رسمی آزمون است، اما داخل هر سرفصل مطالب را **از ساده به سخت** چیده‌ایم. هر درس سه لایه دارد:

- **یاد بگیر:** توضیح ساده‌ی مفهوم، تشبیه روزمره و اصطلاحات کلیدی.
- **اجرا کن:** فرمان‌ها با ساختار کلی، جدول سوییچ‌ها، مثال‌های ترمینال و در پایان لابراتوار گام‌به‌گام با راه‌حل.
- **مرور کن:** نکته‌های آزمون و تمرین‌های پایان هر لابراتوار؛ پاسخ تشریحی همه‌ی تمرین‌ها در پیوست آخر همین جلد آمده است.

نقشه‌ی دو جلد

جلد	فصل‌ها	محتوای اصلی
جلد اول	آمادگی، سرفصل ۱، سرفصل ۲، سرفصل ۳ (بخش اول)	VMware و نصب Ubuntu، تاریخچه و توزیع‌ها، apt و dpkg، مجوزها، SSH، Bash، راهنما، ls و cd، مدیریت فایل، find، مشاهده‌ی فایل، Redirect و Pipe، grep و Regex، آرشیو
جلد دوم	سرفصل ۳ (بخش دوم)، سرفصل ۴، سرفصل ۵	پردازش متن، nano و vim، اسکریپت Bash، سخت‌افزار و دیسک، FHS، فرایند و سرویس و لاگ، شبکه، netplan و networkd، کاربر و گروه، sudo، مجوزها و لینک‌ها، پروژه و آزمون آزمایشی

010-160
کد آزمون · نسخه‌ی ۱٫۶

60′ · 40
۴۰ سؤال در ۶۰ دقیقه

500/800
نمره‌ی قبولی

LTS 24.04
Ubuntu روی VMware

این جزوه برای خودآموزی هم مناسب است

اگر کلاس ندارید، هر روز یک درس بخوانید و لابراتوار همان درس را کامل اجرا کنید. هر خطی را که نمی‌فهمید **تایپ کنید، نه کپی**؛ انگشت‌هایتان باید فرمان‌ها را یاد بگیرند. قبل از هر لابراتوار از ماشین مجازی Snapshot بگیرید تا اگر اشتباهی کردید با یک کلیک برگردید.

راهنمای رنگ‌ها و کادرها

How to read this book

در این جزوه رنگ فقط تزئین نیست؛ هر رنگ به شما می‌گوید کجای مسیر هستید. رنگ سرفصل‌ها از نمادهای آشنای دنیای لینوکس گرفته شده: زرد نوک و پای پنگوئن Tux، نارنجی Ubuntu، سبز اعلان ترمینال، آبی شبکه و بنفش امنیت.

0 — آماده‌سازی
ساخت آزمایشگاه روی VMware Workstation و نصب Ubuntu

1 — جامعه‌ی لینوکس و متن‌باز
تاریخچه، توزیع‌ها، نرم‌افزار و بسته، مجوزها، SSH

2 — پیدا کردن راه در سیستم
Bash، راهنما، فایل و دایرکتوری، find

3 — قدرت خط فرمان
Pipe و grep، پردازش متن، آرشیو، ویرایشگر، اسکریپت

4 — سیستم‌عامل و شبکه
سخت‌افزار، فرایند و لاگ، IP، netplan (جلد دوم)

5 — امنیت و مجوزها
کاربر و گروه، sudo، chmod، فایل‌های ویژه (جلد دوم)

کادرهای داخل درس‌ها

هر کادر رنگ و نماد ثابتی دارد تا با یک نگاه بدانید با چه نوع مطلبی روبه‌رو هستید:

≈ تشبیه
یک مفهوم سخت را با مثالی از زندگی روزمره توضیح می‌دهد. اگر مفهومی برایتان مبهم بود، اول این کادر را بخوانید.

! نکته
میانبر، روش بهتر یا تجربه‌ی عملی که در کار روزمره وقت شما را ذخیره می‌کند.

⚠ هشدار
جایی که یک اشتباه کوچک گران تمام می‌شود؛ مثل حذف فایل بدون بازگشت یا قطع‌شدن شبکه‌ی سرور.

؟ نکته‌ی آزمون

چیزی که در آزمون `010-160` زیاد پرسیده می‌شود. پیش از آزمون همه‌ی این کادرها را دوباره مرور کنید.

VM VMware Workstation

تنظیم یا رفتاری که مخصوص محیط مجازی VMware Workstation است.

✓ بررسی نتیجه (Verify)

در پایان هر لابراتوار می‌گوید از کجا بفهمید کار را درست انجام داده‌اید.

قرارداد نوشتاری ترمینال‌ها

خطی که با علامت سبز `$` شروع می‌شود فرمانی است که با کاربر عادی تایپ می‌کنید و علامت قرمز `#` یعنی شِل root. خطوط کم‌رنگ خروجی نمونه‌اند (ممکن است روی سیستم شما کمی فرق کنند) و متن بنفش بعد از `##` توضیح است و تایپ نمی‌شود. نام کاربر و ماشین در نوار بالای هر ترمینال آمده است. علامت `$` و `#` را هرگز تایپ نکنید؛ فقط بخش سفید را بنویسید.

student@ubuntu-lab: ~

```
$ sudo apt install tree  ## the command you type
Reading package lists... Done
# whoami  ## a root shell
root
```

سرفصل‌های رسمی و نقشه‌ی جزوه

LPI Linux Essentials 1.6 · Exam Objectives

وزن هر سرفصل یعنی تعداد تقریبی سؤال‌های آن در آزمون؛ جمع وزن‌ها ۴۰ است. ستون آخر نشان می‌دهد هر سرفصل در کدام جلد و کدام لابراتوار تمرین می‌شود. سرفصل‌های پروزن (۲٫۱، ۳٫۲، ۳٫۳ و ۴٫۳) بیشترین سؤال را دارند؛ روی آن‌ها بیشتر وقت بگذارید.

کد	سرفصل	وزن	جلد · لابراتوار
1.1	تکامل لینوکس و سیستم‌عامل‌های محبوب	۲	جلد ۱ · LAB 01
1.2	نرم‌افزارهای مهم متن‌باز و مدیریت بسته	۲	جلد ۱ · LAB 02
1.3	نرم‌افزار متن‌باز و مجوزها	۱	جلد ۱ · LAB 03
1.4	مهارت‌های ICT و کار در لینوکس	۲	جلد ۱ · LAB 04
2.1	مبانی خط فرمان	۳	جلد ۱ · LAB 05
2.2	استفاده از خط فرمان برای گرفتن کمک	۲	جلد ۱ · LAB 06
2.3	دایرکتوری‌ها و فهرست فایل‌ها	۲	جلد ۱ · LAB 07
2.4	ساخت، جابه‌جایی و حذف فایل‌ها	۲	جلد ۱ · LAB 08 · 09
3.1	آرشیو و فشرده‌سازی فایل‌ها	۲	جلد ۱ · LAB 13
3.2	جستجو و استخراج داده از فایل‌ها	۳	جلد ۱ · LAB 10 · 11 · 12
3.3	تبدیل دستورات به اسکریپت (+ ویرایشگرها)	۴	جلد ۲
4.1	انتخاب سیستم‌عامل	۱	جلد ۲
4.2	شناخت سخت‌افزار رایانه	۲	جلد ۲
4.3	محل ذخیره‌ی داده‌ها، فرایندها و لاگ‌ها	۳	جلد ۲
4.4	رایانه در شبکه + netplan و networkd	۲	جلد ۲
5.1	امنیت پایه و انواع کاربران	۲	جلد ۲
5.2	ساخت کاربران و گروه‌ها	۲	جلد ۲
5.3	مدیریت مجوزها و مالکیت فایل‌ها	۲	جلد ۲
5.4	فایل‌ها و دایرکتوری‌های ویژه	۱	جلد ۲

نکته‌ی آزمون

آزمون `010-160` چهل سؤال چندگزینه‌ای و جای‌خالی دارد و ۶۰ دقیقه وقت می‌دهد. نمره از ۲۰۰ تا ۸۰۰ است و ۵۰۰ قبولی است. این گواهی پیش‌نیاز رسمی LPIC-1 نیست، اما بهترین نقطه‌ی شروع برای آن است.

مسیر بعد از این دوره

Linux Essentials ← LPIC-1 (مدیر سیستم لینوکس) ← LPIC-2 (مهندس سیستم لینوکس) ← LPIC-3 (تخصص‌های سازمانی). در کنار آن‌ها گواهی‌هایی مثل RHCSA و CKA هم در بازار کار ایران و جهان ارزشمندند.

برنامه‌ی زمانی پیشنهادی دوره

7 weeks · 14 sessions · 3 hours each

هر جلسه حدود یک ساعت تدریس مفاهیم و دو ساعت کار عملی دارد. تمرین‌های پایان هر لابراتوار، تکلیف همان جلسه است. جلسه‌های ۱ تا ۷ از جلد اول و جلسه‌های ۸ تا ۱۴ از جلد دوم تدریس می‌شوند.

هفته	جلسه	موضوع	جلد
۱	۱	ساخت آزمایشگاه VMware، نصب Ubuntu، آشنایی با میزکار و ترمینال	جلد ۱ · فصل ۰
۱	۲	تاریخچه، توزیع‌ها، نرم‌افزارهای متن‌باز، apt و dpkg و snap، مجوزها	جلد ۱ · فصل ۱
۲	۳	مهارت‌های ICT و SSH، مبانی Bash، متغیر و نقل‌قول، PATH و history	جلد ۱ · فصل ۱ و ۲
۲	۴	man و help، درخت فایل، مسیر مطلق و نسبی، pwd و cd و ls	جلد ۱ · فصل ۲
۳	۵	touch و mkdir و cp و mv و rm، Globbing، find و locate	جلد ۱ · فصل ۲
۳	۶	cat و less و head و tail، Redirect و Pipe و tee	جلد ۱ · فصل ۳
۴	۷	grep و Regex، آرشیو و فشرده‌سازی با tar و gzip و zip	جلد ۱ · فصل ۳
۴	۸	cut و sort و uniq و wc و tr و sed و awk، vim و nano	جلد ۲ · فصل ۳
۵	۹	اسکریپت‌نویسی Bash: متغیر، شرط، حلقه و کد خروج	جلد ۲ · فصل ۳
۵	۱۰	انتخاب سیستم‌عامل، سخت‌افزار، دیسک و فایل‌سیستم، FHS	جلد ۲ · فصل ۴
۶	۱۱	فرایندها، systemd، لاگ‌ها؛ مفاهیم شبکه و ip و ss	جلد ۲ · فصل ۴
۶	۱۲	تنظیم IP با netplan و networkd، عیب‌یابی شبکه	جلد ۲ · فصل ۴
۷	۱۳	کاربران و گروه‌ها، sudo، مجوزها، umask، فایل‌های ویژه و لینک‌ها	جلد ۲ · فصل ۵
۷	۱۴	پروژه‌ی پایانی و آزمون آزمایشی	جلد ۲ · پیوست

VMware Workstation

پیش از هر لابراتوار از ماشین مجازی یک Snapshot بگیرید: منوی **VM ← Snapshot ← Take Snapshot**. نام آن را مثل `before-lab05` بگذارید تا هر اشتباهی با یک کلیک قابل جبران باشد.

روش مطالعه‌ی پیشنهادی

۱) متن درس را یک‌بار بدون ترمینال بخوانید. ۲) مثال‌ها را خط‌به‌خط تایپ کنید و خروجی را با جزوه مقایسه کنید. ۳) لابراتوار را اول بدون نگاه به راه‌حل امتحان کنید. ۴) تمرین‌ها را بنویسید و بعد با پاسخ‌نامه مقایسه کنید.

فهرست مطالب جلد اول

Contents · Volume 1

فصل ۰ · آماده‌سازی آزمایشگاه ۱۱

0.1 مجازی‌سازی و VMware Workstation ۱۲

VM مشخصات پیشنهادی ماشین مجازی ۱۴

0.2 ساخت ماشین مجازی در VMware ۱۶

0.3 نصب Ubuntu 24.04 LTS ۱۸

0.4 اولین ورود، میزکار GNOME و ترمینال ۲۰

LAB 00 ساخت آزمایشگاه پایه ۲۲

فصل ۱ · جامعه‌ی لینوکس و مسیر شغلی در متن‌باز ۲۴

1.1 تکامل لینوکس و سیستم‌عامل‌های محبوب ۲۵

1.1 خانواده‌های توزیع لینوکس ۲۷

uname شناسایی سیستم: uname، os-release، lsb_release، hostnamectl ۲۹

LAB 01 تشخیص توزیع، نسخه و خانواده‌ی سیستم ۳۱

1.2 نرم‌افزارهای مهم متن‌باز و مدیریت بسته ۳۳

1.2 مدیریت بسته در Ubuntu: مفاهیم ۳۵

apt فرمان apt: نصب، حذف و به‌روزرسانی ۳۷

dpkg فرمان dpkg و بسته‌های snap ۴۰

LAB 02 نصب، بررسی و حذف یک بسته ۴۲

1.3 نرم‌افزار متن‌باز و مجوزها ۴۴

1.3 بررسی مجوز بسته‌ها در Ubuntu ۴۶

LAB 03 ممیزی مجوز بسته‌ها ۴۷

1.4 مهارت‌های ICT، حریم خصوصی و SSH ۴۸

ssh اتصال امن راه دور با SSH ۴۹

LAB 04 اولین اتصال SSH ۵۰

فصل ۲ · پیدا کردن راه در سیستم لینوکس ۵۲

2.1 مبانی خط فرمان و Bash ۵۳

Bash انواع فرمان: Builtin، External، Alias ۵۵

$VAR متغیرها، متغیرهای محیطی و PATH ۵۷

Quote نقل‌قول و Escape ۵۹

history تاریخچه، میانبرها و ترکیب فرمان‌ها ۶۰

LAB 05 شناخت Shell، متغیر، نقل‌قول و تاریخچه ۶۲
2.2 گرفتن کمک از خط فرمان ۶۴
man صفحات راهنمای man ۶۵
info help، help--، info و مستندات بسته‌ها ۶۷
LAB 06 کار با راهنمای سیستم ۶۸
2.3 دایرکتوری‌ها و فهرست فایل‌ها ۷۰
find جستجوی فایل با find ۷۲
locate جستجوی سریع با locate ۷۴
LAB 07 پیمایش درخت فایل و مقایسه‌ی ls با find ۷۵
2.4 ساخت، کپی، جابه‌جایی و حذف ۷۶
Globbing Globbing: الگوهای Wildcard ۷۸
LAB 08 مدیریت فایل و Globbing ۷۹
2.5 خلاصه فصل دو و تمرین پیشرفته ۸۰
جلد دوم: قدرت پیشرفته ۸۳

BEFORE YOU START · LAB SETUP

0

آماده‌سازی آزمایشگاه

BUILDING YOUR LINUX LAB ON VMWARE WORKSTATION

یک ماشین مجازی امن و قابل تکرار برای تمام تمرین‌ها؛ از ساخت VM تا نصب Ubuntu 24.04 LTS، اولین ورود، اولین ترمینال و Snapshot پایه

وزن در آزمون: پیش‌نیاز عملی

0.1 مجازی‌سازی و VMware Workstation

0.2 ساخت ماشین مجازی

0.3 نصب Ubuntu 24.04 LTS

0.4 اولین ورود، میزکار و ترمینال

LAB 00 ساخت آزمایشگاه پایه

Objective 0.1

وزن در آزمون: پیش‌نیاز

مجازی‌سازی و VMware Workstation

Virtualization Basics

قبل از اینکه حتی یک فرمان لینوکس بزنیم، به یک **محیط امن برای تمرین** نیاز داریم. اگر روی سیستم اصلی خودتان تمرین کنید، یک فرمان اشتباه مثل حذف یک پوشه‌ی سیستمی یا خراب‌کردن تنظیمات شبکه می‌تواند کارتان را مختل کند. راه‌حل حرفه‌ای، **ماشین مجازی** (Virtual Machine یا VM) است.

ماشین مجازی یک کامپیوتر کامل است که به‌صورت نرم‌افزاری داخل کامپیوتر واقعی شما اجرا می‌شود. پردازنده، حافظه، دیسک و کارت شبکه‌ی آن مجازی‌اند، اما سیستم‌عاملی که روی آن نصب می‌کنید فکر می‌کند روی یک سخت‌افزار واقعی است. نرم‌افزاری که این کار را انجام می‌دهد **Hypervisor** نام دارد و در این دوره از **VMware Workstation** استفاده می‌کنیم.

> ≈ **تشبیه**
>
> کامپیوتر واقعی شما مثل یک ساختمان است و هر ماشین مجازی یک آپارتمان داخل آن. هر آپارتمان در، قفل، آشپزخانه و برق جداگانه دارد؛ اگر در یک آپارتمان آتش‌سوزی شود، ساختمان سر جایش است. Snapshot هم مثل یک عکس فوری از آپارتمان است که هر وقت خواستید، آپارتمان را دقیقاً به همان حالت برمی‌گرداند.

Host	کامپیوتر واقعی شما که VMware روی آن نصب است (معمولاً Windows).
Guest	سیستم‌عاملی که داخل ماشین مجازی اجرا می‌شود؛ در این دوره Ubuntu 24.04 LTS.
Hypervisor	نرم‌افزار مجازی‌سازی. VMware Workstation از نوع ۲ است، یعنی روی یک سیستم‌عامل دیگر اجرا می‌شود.
ISO	فایل تصویر دیسک نصب؛ مثل یک DVD نصب در قالب یک فایل.
Snapshot	عکس لحظه‌ای از وضعیت کامل VM (دیسک، تنظیمات و حتی حافظه) برای بازگشت سریع.
VMware Tools	درایورها و ابزارهای بهبود کارایی داخل Guest؛ در Ubuntu بسته‌ی `open-vm-tools` همین نقش را دارد.

چرا VMware Workstation؟

VMware Workstation روی Windows و Linux نصب می‌شود و برای آموزش امکانات خیلی خوبی دارد: Snapshot چندمرحله‌ای، Clone گرفتن از ماشین، شبکه‌های مجازی قابل تنظیم با **Virtual Network Editor** و پشتیبانی پایدار از Ubuntu. از نسخه‌ی ۱۷ به بعد، نسخه‌ی شخصی آن برای استفاده‌ی غیرتجاری رایگان است.

VM

VMware Workstation

قبل از ساخت ماشین، مطمئن شوید قابلیت مجازی‌سازی سخت‌افزاری (Intel VT-x یا AMD-V) در BIOS/UEFI کامپیوتر شما فعال است. اگر هنگام روشن‌کردن VM پیغامی مثل «This host supports Intel VT-x, but Intel VT-x is disabled» دیدید، باید آن را از تنظیمات BIOS فعال کنید.

VM

مشخصات پیشنهادی ماشین مجازی

Recommended VM Hardware

مشخصات زیر برای همه‌ی لابراتوارهای دو جلد کافی است. اگر کامپیوتر شما ضعیف‌تر است، می‌توانید RAM را به ۲ گیگابایت کم کنید، اما محیط گرافیکی کمی کند می‌شود.

مولفه	مقدار پیشنهادی	دلیل
نرم‌افزار مجازی‌سازی	VMware Workstation 17 یا جدیدتر	Snapshot و شبکه‌ی مجازی پایدار
سیستم‌عامل Guest	Ubuntu 24.04 LTS Desktop	مناسب آموزش هم‌زمان GUI و ترمینال
پردازنده	۲ هسته (vCPU)	کافی برای تمرین‌های مقدماتی
حافظه	۴ گیگابایت RAM	روان برای GNOME؛ حداقل ۲ گیگابایت
دیسک	۳۰ گیگابایت، Thin (بدون پیش‌تخصیص)	فضای کافی برای به‌روزرسانی و Snapshot
کارت شبکه‌ی اول	NAT	دسترسی به اینترنت برای نصب بسته‌ها
کارت شبکه‌ی دوم	Host-Only	شبکه‌ی آزمایشگاهی برای تمرین IP ثابت
نام ماشین (hostname)	`ubuntu-lab`	خروجی‌ها برای همه یکدست می‌شود
کاربر	`student` ، عضو گروه sudo	کار روزمره بدون ورود مستقیم با root

انواع شبکه در VMware

یکی از مهم‌ترین تنظیمات VM، نوع کارت شبکه است. VMware سه حالت اصلی دارد و ما در فصل ۴ (جلد دوم) از دو حالت آن برای تمرین netplan استفاده می‌کنیم:

حالت	شبکه‌ی پیش‌فرض VMware	رفتار	کاربرد در این دوره
NAT	VMnet8	VM پشت Host به اینترنت می‌رود؛ از بیرون دیده نمی‌شود	کارت اول، برای apt و اینترنت
Host-Only	VMnet1	فقط Host و VMها هم را می‌بینند؛ اینترنت ندارد	کارت دوم، برای IP ثابت و SSH
Bridged	VMnet0	VM مثل یک کامپیوتر واقعی در شبکه‌ی محل شما IP می‌گیرد	در کلاس معمولاً استفاده نمی‌کنیم

تشبیه

NAT مثل این است که VM پشت یک منشی نشسته: خودش می‌تواند به بیرون زنگ بزند اما کسی از بیرون شماره‌ی مستقیمش را ندارد. Host-Only یک اتاق جلسه‌ی دربسته است که فقط شما و ماشین‌های مجازی‌تان داخلش هستید. Bridged یعنی VM مستقیم کنار بقیه‌ی کامپیوترهای شبکه می‌نشیند.

VMware Workstation

بازه‌ی IP شبکه‌های VMnet1 و VMnet8 روی هر کامپیوتر ممکن است فرق کند. آن را از منوی **Edit ← Virtual Network Editor** ببینید. در این جزوه فرض کرده‌ایم Host-Only روی `192.168.56.0/24` است؛ اگر روی سیستم شما مثلاً `192.168.80.0/24` است، در لابراتوارها همان را جایگزین کنید.

Objective 0.2

وزن در آزمون: پیش‌نیاز

ساخت ماشین مجازی در VMware

Creating the Virtual Machine

فایل ISO رسمی Ubuntu 24.04 LTS Desktop را از سایت `ubuntu.com/download/desktop` دریافت کنید. نام فایل چیزی شبیه `ubuntu-24.04.x-desktop-amd64.iso` است. حالا ماشین را مرحله‌به‌مرحله می‌سازیم.

۱. VMware Workstation را باز کنید و از منوی **File ← New Virtual Machine** را بزنید. حالت **Typical (recommended)** را انتخاب و Next کنید.

۲. گزینه‌ی **Installer disc image file (iso)** را انتخاب و فایل ISO را با Browse پیدا کنید. اگر VMware گزینه‌ی Easy Install را پیشنهاد داد، برای کنترل بیشتر آن را رد کنید یا بعداً با نصب دستی پیش بروید.

۳. در صفحه‌ی Guest Operating System گزینه‌ی **Linux** و نسخه‌ی **Ubuntu 64-bit** را انتخاب کنید.

۴. نام ماشین را `ubuntu-lab` بگذارید و مسیر ذخیره را روی درایوی با فضای کافی (حداقل ۴۰ گیگابایت آزاد) انتخاب کنید.

۵. اندازه‌ی دیسک را **30 GB** بگذارید. گزینه‌ی **Split virtual disk into multiple files** برای جابه‌جایی آسان‌تر مناسب است.

۶. روی **Customize Hardware** بزنید: Memory را 4096 MB، Processors را 2 و Network Adapter را **NAT** بگذارید.

۷. با دکمه‌ی **Add** یک Network Adapter دوم اضافه و آن را روی **Host-only** تنظیم کنید. Close و سپس Finish را بزنید.

نکته

اگر در کلاس هستید، مدرس می‌تواند یک ماشین تمیز بسازد و از آن با **VM ← Manage ← Clone** یک Full Clone برای هر دانشجو تهیه کند. این‌طوری همه از نقطه‌ی شروع یکسان کار می‌کنند.

هشدار

فایل‌های ماشین مجازی را روی فلش یا درایو شبکه‌ی کند نگذارید. VM روی دیسک SSD بسیار روان‌تر است و Snapshotها هم سریع‌تر ساخته می‌شوند.

ساختار فایل‌های یک ماشین مجازی

پس از ساخت، در پوشه‌ی ماشین چند فایل می‌بینید. لازم نیست آن‌ها را دستکاری کنید، اما بهتر است بشناسید:

پسوند	نقش
.vmx	فایل تنظیمات ماشین (سخت‌افزار مجازی)؛ متنی است
.vmdk	دیسک مجازی؛ اطلاعات واقعی سیستم‌عامل اینجاست
.vmsn و .vmsd	اطلاعات Snapshotها

پسوند	نقش
.nvram	تنظیمات BIOS/UEFI مجازی
.log	گزارش اجرای ماشین؛ برای عیب‌یابی VMware

Objective 0.3

وزن در آزمون: پیش‌نیاز

نصب Ubuntu 24.04 LTS

Installing Ubuntu Desktop

ماشین را با **Power on this virtual machine** روشن کنید. پس از بوت از روی ISO، منوی GRUB ظاهر می‌شود؛ گزینه‌ی **Try or Install Ubuntu** را انتخاب کنید. نصب‌کننده‌ی جدید Ubuntu 24.04 (مبتنی بر Flutter) چند صفحه‌ی ساده دارد:

۱ **Language:** زبان English را انتخاب کنید. زبان رابط را انگلیسی نگه دارید تا پیام‌های خطا با جزوه و منابع اینترنتی یکی باشد. زبان فارسی صفحه‌کلید را بعداً اضافه می‌کنیم.

۲ **Accessibility و Keyboard layout:** صفحه‌کلید English (US) را انتخاب کنید.

۳ **Connect to the internet:** چون کارت NAT دارید، اتصال سیمی خودکار برقرار می‌شود.

۴ **Try or Install:** گزینه‌ی **Install Ubuntu** و سپس **Interactive installation** را بزنید.

۵ **Applications:** حالت **Default selection** کافی است. گزینه‌های نصب درایورهای شخص ثالث و فرمت‌های رسانه‌ای را هم می‌توانید تیک بزنید.

۶ **Disk setup:** گزینه‌ی **Erase disk and install Ubuntu** را انتخاب کنید. نگران نباشید؛ این فقط دیسک مجازی ۳۰ گیگابایتی را پاک می‌کند، نه دیسک کامپیوتر شما.

۷ **Create your account:** نام `student`، نام کامپیوتر `ubuntu-lab` و یک رمز قابل‌یادآوری انتخاب کنید. گزینه‌ی **Require my password to log in** را فعال نگه دارید.

۸ **Time zone:** منطقه‌ی Tehran را انتخاب و نصب را شروع کنید. پس از پایان، **Restart now** را بزنید و اگر خواست، Enter را برای خارج‌کردن رسانه‌ی نصب فشار دهید.

چرا LTS؟

LTS یعنی Long Term Support. نسخه‌های LTS اوبونتو هر دو سال یک‌بار (آوریل سال‌های زوج) منتشر می‌شوند و ۵ سال به‌روزرسانی امنیتی استاندارد می‌گیرند؛ با اشتراک Ubuntu Pro این زمان بیشتر هم می‌شود. نام رمز نسخه‌ی 24.04 **Noble Numbat** است و در فایل‌ها با `noble` می‌بینید. عدد 24.04 یعنی سال ۲۰۲۴، ماه ۰۴.

هشدار

رمز کاربر `student` را فراموش نکنید. این کاربر اولین کاربر سیستم است و به‌طور خودکار عضو گروه `sudo` می‌شود؛ یعنی همین رمز برای کارهای مدیریتی هم پرسیده می‌شود. در Ubuntu کاربر root به‌طور پیش‌فرض رمز ندارد و ورود مستقیم با آن بسته است.

پس از اولین بوت

بعد از راه‌اندازی مجدد، صفحه‌ی ورود GDM را می‌بینید. وارد شوید. ویزارد خوش‌آمدگویی چند سؤال درباره‌ی Ubuntu Pro و ارسال گزارش می‌پرسد که می‌توانید رد کنید. معمولاً بسته‌ی `open-vm-tools-desktop` به‌طور خودکار نصب است و با آن، اندازه‌ی صفحه با پنجره‌ی VMware هماهنگ می‌شود و Copy/Paste بین Host و Guest کار می‌کند. اگر کار نکرد، در LAB 00 آن را نصب می‌کنیم.

Objective 0.4

وزن در آزمون: پیش‌نیاز

اولین ورود، میزکار GNOME و ترمینال

First Login, GNOME and the Terminal

محیط گرافیکی پیش‌فرض Ubuntu، **GNOME** است. در سمت چپ نوار **Dock** با برنامه‌های پرکاربرد، در بالا نوار وضعیت با ساعت و منوی سیستم، و در گوشه‌ی پایین‌چپ دکمه‌ی **Show Apps** را می‌بینید. اما قهرمان این دوره، برنامه‌ای به نام **Terminal** است.

ترمینال پنجره‌ای است که در آن به‌جای کلیک‌کردن، **فرمان تایپ می‌کنید**. برنامه‌ای که داخل ترمینال فرمان شما را می‌خواند و اجرا می‌کند **Shell** نام دارد و شِل پیش‌فرض Ubuntu، **Bash** است. در فصل ۲ مفصل با آن آشنا می‌شویم؛ فعلاً فقط باید بدانید چطور ترمینال را باز کنید و چند کلید میانبر حیاتی را بشناسید.

کلیدهای میانبر ضروری

کلید	کار
Ctrl+Alt+T	باز کردن ترمینال جدید
Ctrl+Shift+C و Ctrl+Shift+V	کپی و چسباندن داخل ترمینال (Ctrl+C در ترمینال یعنی توقف فرمان!)
Ctrl+Shift+T	باز کردن تب جدید در ترمینال
Ctrl+Shift+ + و Ctrl+ -	بزرگ و کوچک‌کردن متن ترمینال
Super (کلید ویندوز)	نمای Activities و جستجوی برنامه‌ها
Ctrl+Alt (در VMware)	آزاد کردن ماوس و صفحه‌کلید از داخل VM
Ctrl+Alt+Enter (در VMware)	حالت تمام‌صفحه

> ⚠ **هشدار**
>
> در ترمینال، کلید **Ctrl+C** به معنی کپی نیست؛ فرمانِ در حال اجرا را متوقف می‌کند. برای کپی از **Ctrl+Shift+C** یا انتخاب متن و کلیک راست استفاده کنید. این اولین تفاوت مهم با Windows است.

اولین فرمان‌ها

ترمینال را با Ctrl+Alt+T باز کنید. خطی مثل `student@ubuntu-lab:~$` می‌بینید که به آن **Prompt** یا اعلان می‌گویند و یعنی «شِل آماده‌ی دریافت فرمان است». چند فرمان بی‌خطر را امتحان کنید و بعد از هر کدام Enter بزنید:

```
student@ubuntu-lab: ~
$ whoami  ## who am I?
student
$ hostname  ## name of this machine
ubuntu-lab
$ date  ## current date and time
Sun Oct  4 11:45:12 AM +0330 2026
$ uptime  ## how long the system has been running
 11:45:20 up 12 min,  1 user,  load average: 0.08, 0.15, 0.10
$ clear  ## clear the screen (or press Ctrl+L)
```

> **نکته**
>
> نوشتن فرمان‌ها را با کلید **Tab** سریع کنید. مثلاً `hostn` را بنویسید و Tab بزنید؛ Bash خودش `hostname` را کامل می‌کند. کلیدهای جهت بالا و پایین هم فرمان‌های قبلی را برمی‌گردانند.

افزودن صفحه‌کلید فارسی

برای نوشتن یادداشت فارسی، از **Settings ← Keyboard ← Input Sources ← Add Input Source** زبان **Persian** را اضافه کنید. جابه‌جایی بین زبان‌ها با **Super+Space** انجام می‌شود. توجه کنید فرمان‌ها همیشه انگلیسی تایپ می‌شوند.

LAB 00

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

ساخت آزمایشگاه پایه

ماشین را بسازید، به‌روزرسانی کنید، ابزارهای پایه را نصب کنید و یک Snapshot پایه بگیرید که همه‌ی لابراتوارهای بعدی از آن شروع می‌شوند.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱ طبق درس ۰٫۲ ماشینی با نام `ubuntu-lab`، ۲ هسته، ۴ گیگابایت RAM، ۳۰ گیگابایت دیسک، کارت اول NAT و کارت دوم Host-Only بسازید.

۲ طبق درس ۰٫۳ Ubuntu 24.04 LTS را نصب کنید و کاربر `student` را بسازید.

۳ ترمینال را باز کنید، فهرست بسته‌ها را به‌روز کنید و سیستم را کامل ارتقا دهید.

۴ ابزارهای پایه‌ی دوره (vim، curl، tree، git، openssh-server و...) را نصب کنید.

۵ نسخه‌ی سیستم‌عامل، نام ماشین و کارت‌های شبکه را بررسی کنید.

۶ ماشین را خاموش کنید و یک Snapshot با نام `clean-install` بگیرید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ sudo apt update  ## refresh package lists (asks for your password)
[sudo] password for student:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Reading package lists... Done
$ sudo apt full-upgrade -y  ## install all available updates
$ sudo apt install -y vim nano curl wget tree git openssh-server net-tools dnsutils lsof
plocate open-vm-tools-desktop
$ hostnamectl  ## OS, kernel and virtualization info
 Static hostname: ubuntu-lab
Operating System: Ubuntu 24.04.1 LTS
          Kernel: Linux 6.8.0-45-generic
  Virtualization: vmware
$ ip -br addr  ## brief list of network interfaces
lo        UNKNOWN  127.0.0.1/8 ::1/128
ens33     UP       192.168.17.128/24
ens37     UP       192.168.56.128/24
$ sudo poweroff
```

نکته

هنگام تایپ رمز بعد از `sudo` هیچ ستاره یا نقطه‌ای نمایش داده نمی‌شود. این باگ نیست؛ رمز را کامل تایپ کنید و Enter بزنید.

VM **VMware Workstation**

پس از خاموش‌شدن، از منوی **VM** ← **Snapshot** ← **Take Snapshot** نام `clean-install` و توضیح «نصب تمیز با ابزارهای پایه» را وارد کنید. برای بازگشت در آینده از **Snapshot Manager** (کلید Ctrl+M) استفاده کنید.

✓ **بررسی نتیجه (Verify)**

خروجی `hostnamectl` باید Ubuntu 24.04 LTS و `Virtualization: vmware` را نشان دهد. باید دو کارت شبکه (معمولاً `ens33` و `ens37`) در وضعیت UP ببینید. کارت NAT معمولاً IP در بازه‌ی `192.168.x.x` یا `172.16.x.x` می‌گیرد؛ روی IP نمونه قفل نشوید، مهم وجود IP و وضعیت UP است.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. چرا قبل از هر لابراتوار Snapshot می‌گیریم؟
2. تفاوت شبکه‌ی NAT و Host-Only در VMware چیست؟
3. چرا برای کار روزمره با root وارد نمی‌شویم؟
4. در ترمینال برای کپی و چسباندن متن از چه کلیدهایی استفاده می‌شود و Ctrl+C چه می‌کند؟
5. عدد 24.04 در نام نسخه‌ی Ubuntu چه معنایی دارد؟

EXAM 010-160 · TOPIC 1

1

جامعه‌ی لینوکس و مسیر شغلی در متن‌باز

THE LINUX COMMUNITY AND A CAREER IN OPEN SOURCE

از تاریخچه‌ی Unix و GNU تا توزیع‌ها، نرم‌افزارهای متن‌باز، مدیریت بسته، مجوزها و مهارت‌های ICT؛ پایه‌ی فکری همه‌ی فصل‌های بعد

وزن در آزمون: ۷ سؤال از ۴۰

1.1 تکامل لینوکس و توزیع‌ها

1.2 نرم‌افزارهای متن‌باز و مدیریت بسته

1.3 نرم‌افزار متن‌باز و مجوزها

1.4 مهارت‌های ICT، حریم خصوصی و SSH

Objective 1.1

وزن در آزمون: ۲

تکامل لینوکس و سیستم‌عامل‌های محبوب

Linux Evolution and Popular Operating Systems

سیستم‌عامل نرم‌افزاری است که بین شما (و برنامه‌هایتان) و سخت‌افزار قرار می‌گیرد. وقتی در یک برنامه دکمه‌ی ذخیره را می‌زنید، برنامه خودش مستقیم با دیسک حرف نمی‌زند؛ از سیستم‌عامل می‌خواهد فایل را بنویسد. Windows، macOS، Android و Linux همگی سیستم‌عامل‌اند.

لینوکس در اصل نام یک کرنل است، نه کل سیستم‌عامل. کرنل (Kernel) هسته‌ی سیستم‌عامل است و کارهای اساسی را انجام می‌دهد: تقسیم زمان CPU بین برنامه‌ها، مدیریت حافظه، صحبت با دستگاه‌ها از طریق درایورها، مدیریت فایل‌سیستم و شبکه. اما کرنل به‌تنهایی به درد کاربر نمی‌خورد؛ به شِل، ابزارهای خط فرمان، کتابخانه‌ها، مدیر بسته و محیط گرافیکی هم نیاز داریم. مجموعه‌ی آماده‌ی همه‌ی این‌ها را **توزیع** (Distribution) می‌گویند.

> ≈ **تشبیه**
>
> کرنل مثل موتور خودرو است. موتور به‌تنهایی شما را جایی نمی‌برد؛ بدنه، فرمان، صندلی و داشبورد لازم است. شرکت‌های مختلف با یک موتور مشترک، خودروهای متفاوتی می‌سازند. توزیع‌های لینوکس هم با یک کرنل مشترک، سیستم‌عامل‌های متفاوتی مثل Ubuntu، Fedora و Debian ساخته‌اند.

تاریخچه در یک نگاه

سال	رویداد
۱۹۶۹	Ken Thompson و Dennis Ritchie در آزمایشگاه Bell سیستم‌عامل **Unix** را ساختند؛ بعدها با زبان C بازنویسی شد.
۱۹۸۳	Richard Stallman پروژه‌ی **GNU** (مخفف GNU's Not Unix) را برای ساخت یک سیستم‌عامل کاملاً آزاد شبیه Unix آغاز کرد.
۱۹۸۵	بنیاد نرم‌افزار آزاد (FSF) تأسیس شد؛ مجوز GPL کمی بعد منتشر شد.
۱۹۹۱	**Linus Torvalds**، دانشجوی فنلاندی، نسخه‌ی اول کرنل Linux را منتشر کرد. ابزارهای GNU کنار آن قرار گرفتند و سیستم کامل GNU/Linux شکل گرفت.
۱۹۹۲	کرنل لینوکس تحت مجوز GPLv2 منتشر شد.
۱۹۹۳	توزیع‌های **Slackware** و **Debian** متولد شدند.
۱۹۹۴ تا ۱۹۹۵	**Red Hat** و **SUSE** نسخه‌های تجاری ارائه دادند.
۲۰۰۴	شرکت Canonical اولین نسخه‌ی **Ubuntu** را بر پایه‌ی Debian منتشر کرد.
۲۰۰۸	**Android** با کرنل لینوکس عرضه شد و لینوکس به جیب میلیاردها نفر رفت.

سال	رویداد
امروز	همه‌ی ۵۰۰ ابررایانه‌ی برتر جهان، بیشتر سرورهای ابری و وب، و میلیاردها دستگاه IoT با لینوکس کار می‌کنند.

1.1 خانواده‌های توزیع لینوکس

Linux Distribution Families

صدها توزیع لینوکس وجود دارد، اما بیشتر آن‌ها از چند خانواده‌ی اصلی مشتق شده‌اند. توزیع‌های یک خانواده قالب بسته و مدیر بسته‌ی مشترک دارند؛ اگر یکی را بلد باشید، بقیه را هم به‌سرعت یاد می‌گیرید.

خانواده	توزیع‌های مهم	قالب بسته	مدیر بسته	کاربرد رایج
Debian	Debian، Ubuntu، Linux Mint، Kali	deb.	apt و dpkg	سرور، دسکتاپ، امنیت
Red Hat	RHEL، Fedora، CentOS Stream، Rocky، AlmaLinux	rpm.	dnf (قبلاً yum) و rpm	سرور سازمانی
SUSE	SUSE Linux Enterprise، openSUSE	rpm.	zypper	سرور سازمانی اروپا
Arch	Arch Linux، Manjaro	pkg.tar.zst.	pacman	کاربران حرفه‌ای، Rolling
مستقل/جاسازی‌شده	Android، Raspberry Pi OS، Alpine	متفاوت	متفاوت	موبایل، IoT، کانتینر

Kernel	هسته‌ی سیستم‌عامل؛ زمان CPU، حافظه، درایور دستگاه‌ها و فایل‌سیستم را مدیریت می‌کند.
Distribution	مجموعه‌ی آماده‌ی کرنل، ابزارها، بسته‌ها، نصب‌کننده و تنظیمات پیش‌فرض.
GNU/Linux	نام دقیق سیستم: کرنل Linux به‌علاوه‌ی ابزارهای پروژه‌ی GNU (مثل bash، ls، cp).
LTS	نسخه‌ی پایدار با پشتیبانی طولانی؛ Ubuntu 24.04 LTS با نام رمز noble.
Rolling release	توزیعی که نسخه‌ی ثابت ندارد و مدام به‌روز می‌شود، مثل Arch.
Embedded Linux	لینوکس روی دستگاه‌های کوچک: روتر، تلویزیون هوشمند، دوربین، خودرو.

؟ نکته‌ی آزمون

در آزمون می‌پرسند «Ubuntu از کدام توزیع مشتق شده؟» (Debian) یا «کدام توزیع از خانواده‌ی Red Hat است؟» (Fedora، CentOS، Rocky). همچنین بدانید **Android** از کرنل لینوکس استفاده می‌کند ولی ابزارهای GNU را ندارد، و **Raspberry Pi OS** بر پایه‌ی Debian است.

لینوکس در مقایسه با Windows و macOS

Windows سیستم‌عامل تجاری مایکروسافت است و کد منبع آن بسته است. **macOS** شرکت Apple بر پایه‌ی Darwin ساخته شده که ریشه در BSD (خانواده‌ی Unix) دارد؛ برای همین ترمینال macOS خیلی شبیه لینوکس است. **Linux** متن‌باز است، روی

تقریباً هر سخت‌افزاری اجرا می‌شود و هر کسی می‌تواند توزیع خودش را بسازد. **BSD**ها (FreeBSD، OpenBSD) سیستم‌عامل‌های Unixمانند دیگری هستند که کرنل جداگانه دارند و لینوکس نیستند.

uname

شناسایی سیستم: uname، os-release، hostnamectl ،lsb_release

Identifying Your System

اولین کاری که روی هر سرور ناشناس انجام می‌دهید این است که بفهمید **چه توزیعی، چه نسخه‌ای و چه کرنلی** دارد. چهار ابزار زیر این اطلاعات را می‌دهند.

ساختار کلی · SYNTAX
```
uname [OPTION]...
```

فرمان `uname` اطلاعات کرنل را نشان می‌دهد (نه توزیع). بدون گزینه فقط کلمه‌ی Linux را چاپ می‌کند.

سوییچ‌های رایج `uname`

سوییچ	شکل بلند	کاربرد	مثال
`-a`	`--all`	همه‌ی اطلاعات در یک خط	`uname -a`
`-s`	`--kernel-name`	نام کرنل (Linux)	`uname -s`
`-r`	`--kernel-release`	نسخه‌ی کرنل؛ پرکاربردترین گزینه	`uname -r`
`-m`	`--machine`	معماری سخت‌افزار مثل x86_64 یا aarch64	`uname -m`
`-n`	`--nodename`	نام میزبان در شبکه	`uname -n`
`-o`	`--operating-system`	سیستم‌عامل (GNU/Linux)	`uname -o`

student@ubuntu-lab: ~
```
$ uname
Linux
$ uname -r
6.8.0-45-generic
$ uname -m
x86_64
$ uname -a
Linux ubuntu-lab 6.8.0-45-generic #45-Ubuntu SMP PREEMPT_DYNAMIC ... x86_64 x86_64 x86_64
GNU/Linux
```

فایل /etc/os-release

همه‌ی توزیع‌های مدرن اطلاعات خود را در فایل متنی `/etc/os-release` نگه می‌دارند. فرمان `cat` محتوای فایل را چاپ می‌کند (در فصل ۳ کامل می‌خوانیم):

```
student@ubuntu-lab: ~
$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.1 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
```

خط `ID_LIKE=debian` می‌گوید این توزیع از خانواده‌ی Debian است. روی Rocky Linux همین خط `ID_LIKE="rhel centos fedora"` است.

lsb_release -a

نام، نسخه و codename توزیع (Linux Standard Base = LSB). گزینه‌ی `-d` فقط توضیح و `-c` فقط codename را می‌دهد.

```
$ lsb_release -a
```

hostnamectl

نام میزبان، سیستم‌عامل، کرنل، معماری و نوع مجازی‌سازی؛ همچنین برای تغییر hostname.

```
$ hostnamectl
```

cat /proc/version

نسخه‌ی کرنل و کامپایلری که با آن ساخته شده؛ مستقیم از کرنل.

```
$ cat /proc/version
```

systemd-detect-virt

نوع مجازی‌سازی را می‌گوید؛ روی VMware کلمه‌ی vmware و روی سخت‌افزار واقعی none.

```
$ systemd-detect-virt
```

LAB 01

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

تشخیص توزیع، نسخه و خانواده‌ی سیستم

از خروجی خود سیستم تشخیص دهید با چه کرنل، چه توزیع، چه نسخه و چه خانواده‌ای کار می‌کنید و آیا داخل ماشین مجازی هستید یا نه.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. نسخه‌ی کرنل و معماری پردازنده را با `uname` پیدا کنید.

۲. نام توزیع، نسخه، codename و خانواده (ID_LIKE) را از `/etc/os-release` بخوانید.

۳. همین اطلاعات را با `lsb_release` و `hostnamectl` تأیید کنید.

۴. بررسی کنید سیستم داخل VMware اجرا می‌شود.

۵. در دفترتان جدولی بسازید: سه توزیع از سه خانواده‌ی مختلف و مدیر بسته‌ی هر کدام.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ uname -r; uname -m
6.8.0-45-generic
x86_64
$ grep -E '^(ID|ID_LIKE|VERSION_ID|VERSION_CODENAME)=' /etc/os-release  ## grep: chapter 3
VERSION_ID="24.04"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
$ lsb_release -dc
Description:    Ubuntu 24.04.1 LTS
Codename:       noble
$ hostnamectl | grep -E 'Operating System|Kernel|Virtualization'
  Virtualization: vmware
Operating System: Ubuntu 24.04.1 LTS
          Kernel: Linux 6.8.0-45-generic
$ systemd-detect-virt
vmware
```

نکته

علامت `;` بین دو فرمان یعنی «اولی را اجرا کن، بعد دومی را». علامت `|` (Pipe) خروجی فرمان اول را به ورودی فرمان دوم می‌دهد. هر دو را در فصل ۳ مفصل می‌خوانیم؛ فعلاً فقط از آن‌ها استفاده کنید.

✓ بررسی نتیجه (Verify)

وجود `ID_LIKE=debian` یعنی Ubuntu از خانواده‌ی Debian است و در نتیجه از apt و بسته‌های deb استفاده می‌کند. نسخه‌ی کرنل شما ممکن است عدد دیگری مثل `6.8.0-48` باشد؛ مهم این است که با `6.8` شروع شود.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. تفاوت Kernel و Distribution چیست؟
2. Rocky Linux به کدام خانواده تعلق دارد و چه مدیر بسته‌ای دارد؟
3. فرمان `uname -r` چه چیزی را نشان می‌دهد؟
4. کدام فایل در همه‌ی توزیع‌های مدرن نام و نسخه‌ی توزیع را نگه می‌دارد؟
5. چرا Android را یک سیستم GNU/Linux کامل نمی‌دانند؟

Objective 1.2

وزن در آزمون: ۲

نرم‌افزارهای مهم متن‌باز و مدیریت بسته

Major Open Source Applications

بخش بزرگی از آزمون Linux Essentials درباره‌ی شناختن نرم‌افزارهای مشهور متن‌باز است. لازم نیست همه را کار کرده باشید، اما باید بدانید هر کدام **چه کاری** انجام می‌دهد.

نرم‌افزارهای دسکتاپ

دسته	نرم‌افزار متن‌باز	معادل تجاری آشنا
مجموعه‌ی اداری	LibreOffice (Writer، Calc، Impress)؛ قبلاً OpenOffice	Microsoft Office
مرورگر وب	Mozilla Firefox، Chromium	Chrome، Edge
ایمیل	Thunderbird	Outlook
ویرایش تصویر	GIMP، Inkscape (برداری)	Photoshop، Illustrator
سه‌بعدی و انیمیشن	Blender	3ds Max، Maya
صدا و ویدیو	Audacity، VLC، Kdenlive، OBS Studio	Premiere

نرم‌افزارهای سرور

کاربرد	نرم‌افزار
وب‌سرور	Apache HTTP Server، Nginx
پایگاه داده	MySQL، MariaDB، PostgreSQL، SQLite
اشتراک فایل	Samba (با Windows)، NFS (بین لینوکس‌ها)
ایمیل سرور	Postfix، Dovecot
DNS و DHCP	BIND، dnsmasq، ISC DHCP / Kea
مدیریت محتوا	WordPress، Drupal، Joomla
ابر و کانتینر	OpenStack، Docker، Kubernetes

زبان‌های برنامه‌نویسی

زبان	ویژگی
C	زبان کامپایلی؛ کرنل لینوکس با آن نوشته شده

زبان	ویژگی
Java	کامپایل به bytecode و اجرا روی JVM
Python	تفسیری، محبوب برای اتوماسیون و داده
Perl	تفسیری، قوی در پردازش متن
PHP	سمت سرور برای وب
JavaScript	سمت مرورگر و با Node.js سمت سرور
(Bash) Shell	اتوماسیون کارهای سیستمی؛ در جلد دوم می‌نویسیم

؟ نکته‌ی آزمون

تفاوت زبان **کامپایلی** (C، C++؛ کد یک‌بار به فایل اجرایی تبدیل می‌شود) و **تفسیری** (Python، Perl، Bash؛ مفسر هر بار کد را می‌خواند و اجرا می‌کند) را بدانید. همچنین بدانید LibreOffice از قالب استاندارد **ODF** (پسوندهای .odt، .ods و .odp) استفاده می‌کند.

1.2 مدیریت بسته در Ubuntu: مفاهیم

Package Management Concepts

در Windows معمولاً برنامه را از سایت سازنده دانلود و فایل setup را اجرا می‌کنید. در لینوکس روش اصلی متفاوت است: نرم‌افزارها به‌صورت **بسته** (Package) در **مخزن** (Repository) رسمی توزیع نگهداری می‌شوند و با **مدیر بسته** نصب می‌شوند. مدیر بسته وابستگی‌ها را خودش پیدا و نصب می‌کند، امضای دیجیتال بسته را بررسی می‌کند و بعداً همه را با یک فرمان به‌روز می‌کند.

> **≈ تشبیه**
>
> مخزن مثل یک فروشگاه اپلیکیشن رسمی است که همه‌ی برنامه‌هایش بررسی و امضا شده‌اند. apt فروشنده‌ای است که وقتی برنامه‌ای را می‌خواهید، خودش وسایل جانبی لازم (وابستگی‌ها) را هم می‌آورد. dpkg کارگری است که فقط جعبه‌ای را که دستش می‌دهید باز می‌کند و اگر قطعه‌ای کم باشد، فقط خطا می‌دهد.

Package	یک فایل (مثلاً .deb) شامل برنامه، فایل‌های تنظیمات، مستندات و اطلاعات وابستگی.
Repository	مخزن بسته‌های آزموده و امضاشده؛ آدرس‌ها در `/etc/apt/sources.list.d/ubuntu.sources` .
Dependency	بسته یا کتابخانه‌ای که یک برنامه برای اجرا لازم دارد.
apt	ابزار سطح‌بالا: جستجو، نصب، حذف و به‌روزرسانی با حل خودکار وابستگی.
dpkg	ابزار سطح‌پایین: نصب و بررسی فایل deb محلی؛ وابستگی را خودش دانلود نمی‌کند.
Snap	قالب بسته‌ی مستقل از توزیع که همه‌ی وابستگی‌ها را با خود دارد؛ حجیم‌تر اما ایزوله.
PPA	مخزن شخصی روی Launchpad برای نسخه‌های تازه‌تر؛ فقط از منابع معتبر.

مخازن Ubuntu

مخازن رسمی Ubuntu چهار بخش (Component) دارند: **main** (پشتیبانی رسمی Canonical)، **universe** (نرم‌افزارهای جامعه)، **restricted** (درایورهای غیرآزاد) و **multiverse** (نرم‌افزارهای با محدودیت قانونی یا مجوز). در Ubuntu 24.04 فهرست مخازن در قالب جدید deb822 در فایل زیر است:

```
student@ubuntu-lab: ~
$ cat /etc/apt/sources.list.d/ubuntu.sources
Types: deb
URIs: http://archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

هشدار

از نصب بسته از سایت‌ها و منابع ناشناس خودداری کنید. بسته‌ی deb هنگام نصب با دسترسی root اجرا می‌شود و می‌تواند هر کاری با سیستم بکند. همیشه اول مخزن رسمی، بعد سایت رسمی سازنده.

apt فرمان apt: نصب، حذف و به‌روزرسانی

The apt Command

SYNTAX · ساختار کلی
```
apt [OPTIONS] COMMAND [PACKAGE]...
```

فرمان `apt` ابزار اصلی مدیریت بسته در Ubuntu است. فرمان‌هایی که سیستم را تغییر می‌دهند (نصب، حذف، به‌روزرسانی) به `sudo` نیاز دارند؛ فرمان‌های جستجو و نمایش نه.

زیرفرمان‌های رایج `apt`

سوییچ	شکل بلند	کاربرد	مثال
`update`		فهرست بسته‌های مخازن را تازه می‌کند؛ هیچ برنامه‌ای را ارتقا نمی‌دهد	`sudo apt update`
`upgrade`		همه‌ی بسته‌های قابل ارتقا را به‌روز می‌کند (بدون حذف بسته)	`sudo apt upgrade`
`full-upgrade`		ارتقای کامل؛ در صورت نیاز بسته‌ای را حذف یا اضافه می‌کند	`sudo apt full-upgrade`
`install`		نصب یک یا چند بسته با وابستگی‌ها	`sudo apt install tree htop`
`remove`		حذف بسته؛ فایل‌های تنظیمات می‌ماند	`sudo apt remove tree`
`purge`		حذف بسته به‌همراه فایل‌های تنظیمات	`sudo apt purge tree`
`autoremove`		حذف وابستگی‌هایی که دیگر لازم نیستند	`sudo apt autoremove`
`search`		جستجو در نام و توضیح بسته‌ها	`apt search nginx`
`show`		نمایش اطلاعات کامل بسته	`apt show tree`
`list --installed`		فهرست بسته‌های نصب‌شده	`apt list --installed`
`list --upgradable`		بسته‌هایی که نسخه‌ی جدید دارند	`apt list --upgradable`
`policy`		نسخه‌ی نصب‌شده، نسخه‌ی کاندید و مخزن منبع	`apt policy bash`
`depends`		وابستگی‌های یک بسته	`apt depends tree`

گزینه‌های پرکاربرد

سوییچ	شکل بلند	کاربرد	مثال
`-y`	`--yes`	پاسخ خودکار «بله» به پرسش‌ها؛ مناسب اسکریپت	`sudo apt install -y git`
`-s`	`--simulate`	شبیه‌سازی؛ نشان می‌دهد چه می‌شود بی‌آنکه کاری انجام دهد	`apt install -s nginx`
`--no-install-recommends`		نصب بدون بسته‌های پیشنهادی؛ نصب سبک‌تر	`sudo apt install --no-install-recommends vim`
`--reinstall`		نصب دوباره‌ی بسته‌ای که خراب شده	`sudo apt install --reinstall bash`

```
student@ubuntu-lab: ~
$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Reading package lists... Done
12 packages can be upgraded. Run 'apt list --upgradable' to see them.
$ apt show tree | head -4
Package: tree
Version: 2.1.1-2ubuntu3
Priority: optional
Section: universe/utils
$ sudo apt install tree
The following NEW packages will be installed:
  tree
Do you want to continue? [Y/n] y
```

نکته‌ی آزمون

فرق `apt update` و `apt upgrade` از سؤال‌های پرتکرار است: **update** فقط فهرست را از مخزن می‌گیرد، **upgrade** بسته‌ها را واقعاً ارتقا می‌دهد. همیشه اول update، بعد upgrade. فرق `remove` و `purge` هم مهم است: purge فایل‌های تنظیمات را هم پاک می‌کند.

apt و apt-get

قبل از `apt` ، ابزارهای `apt-get` و `apt-cache` استفاده می‌شدند و هنوز هم وجود دارند. `apt` برای کار تعاملی (خروجی رنگی و نوار پیشرفت) و `apt-get` برای اسکریپت‌ها (خروجی پایدار) توصیه می‌شود. معادل‌ها: `apt-get install` ، `apt-get remove` ، `apt-cache search` ، `apt-cache show` .

dpkg فرمان dpkg و بسته‌های snap

dpkg and snap

```
SYNTAX · ساختار کلی
dpkg [OPTION] PACKAGE-or-FILE
```

`dpkg` ابزار پایه‌ای است که apt در پشت صحنه از آن استفاده می‌کند. برای کار با فایل‌های `.deb` محلی و پرسیدن اینکه «این فایل مال کدام بسته است؟» بسیار کاربردی است.

سوییچ‌های رایج `dpkg`

سوییچ	شکل بلند	کاربرد	مثال
`-i`	`--install`	نصب فایل deb محلی	`sudo dpkg -i app.deb`
`-r`	`--remove`	حذف بسته (تنظیمات می‌ماند)	`sudo dpkg -r app`
`-P`	`--purge`	حذف کامل با تنظیمات	`sudo dpkg -P app`
`-l`	`--list`	فهرست بسته‌های نصب‌شده (با الگو)	`dpkg -l 'open*'`
`-L`	`--listfiles`	فایل‌هایی که یک بسته نصب کرده	`dpkg -L tree`
`-S`	`--search`	این فایل متعلق به کدام بسته است؟	`dpkg -S /bin/ls`
`-s`	`--status`	وضعیت و اطلاعات یک بسته	`dpkg -s bash`

```
student@ubuntu-lab: ~
$ dpkg -S /bin/ls
coreutils: /bin/ls
$ dpkg -L tree | head -4
/.
/usr
/usr/bin
/usr/bin/tree
$ dpkg -l | grep openssh
ii  openssh-server  1:9.6p1-3ubuntu13  amd64  secure shell (SSH) server
```

نکته

اگر فایل deb را با `dpkg -i` نصب کردید و خطای وابستگی گرفتید، فرمان `sudo apt -f install` (fix-broken) وابستگی‌های ناقص را از مخزن نصب می‌کند. روش ساده‌تر: از ابتدا با `sudo apt install ./app.deb` نصب کنید (حتماً با `./`)؛ apt هم فایل محلی را نصب می‌کند و هم وابستگی‌ها را.

Snap

Ubuntu علاوه بر deb از قالب **snap** هم پشتیبانی می‌کند. بسته‌ی snap همه‌ی وابستگی‌هایش را با خود دارد، در محیط ایزوله اجرا می‌شود و به‌طور خودکار به‌روز می‌شود. Firefox در Ubuntu 24.04 به‌صورت پیش‌فرض snap است.

snap find

جستجو در فروشگاه Snap

```
$ snap find vlc
```

snap install

نصب بسته‌ی snap

```
$ sudo snap install vlc
```

snap list

فهرست snapهای نصب‌شده

```
$ snap list
```

snap remove

حذف snap

```
$ sudo snap remove vlc
```

snap refresh

به‌روزرسانی snapها

```
$ sudo snap refresh
```

which

مسیر فایل اجرایی یک فرمان

```
$ which python3
```

LAB 02

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

نصب، بررسی و حذف یک بسته

چرخه‌ی کامل عمر یک بسته را با apt و dpkg تجربه کنید: جستجو، نصب، بررسی فایل‌ها، پیدا کردن مالک یک فایل و حذف.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. فهرست مخازن را تازه کنید و بسته‌ی `tree` و `htop` را جستجو کنید.

۲. اطلاعات بسته‌ی `htop` را با `apt show` ببینید و نسخه‌ی آن را یادداشت کنید.

۳. هر دو بسته را نصب کنید و ساختار خانه‌ی خود را با `tree` ببینید.

۴. با `dpkg -S` بفهمید فایل `/bin/ls` مال کدام بسته است و با `dpkg -L` فایل‌های بسته‌ی tree را ببینید.

۵. بسته‌ی `htop` را با purge حذف کنید و با `which` بررسی کنید که دیگر وجود ندارد.

۶. وابستگی‌های بلااستفاده را پاک کنید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ sudo apt update
$ apt search --names-only '^htop$'
htop/noble 3.3.0-4build1 amd64
  interactive processes viewer
$ apt show htop | grep -E '^(Version|Section|Depends)'
$ sudo apt install -y tree htop
$ tree -L 1 ~
/home/student
├── Desktop
├── Documents
├── Downloads
...
$ dpkg -S /bin/ls
coreutils: /bin/ls
$ dpkg -L tree | grep bin
/usr/bin/tree
$ sudo apt purge -y htop
$ which htop || echo 'htop is removed'  ## run 2nd only if 1st fails
htop is removed
$ sudo apt autoremove -y
```

✓ بررسی نتیجه (Verify)

بعد از نصب، `which tree` باید `/usr/bin/tree` را چاپ کند. بعد از purge، فرمان `dpkg -l htop` یا چیزی پیدا نمی‌کند یا وضعیت را بدون `ii` نشان می‌دهد. علامت `||` یعنی «اگر فرمان اول ناموفق بود، دومی را اجرا کن».

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فرق `apt` و `dpkg` چیست؟
2. `apt update` چه کاری انجام نمی‌دهد؟
3. فرق `apt remove` و `apt purge` چیست؟
4. چطور بفهمیم فایل `/usr/bin/ssh` متعلق به کدام بسته است؟
5. معادل مدیر بسته‌ی apt در Fedora و openSUSE چیست؟

Objective 1.3

وزن در آزمون: ۱

نرم‌افزار متن‌باز و مجوزها

Open Source Software and Licensing

نرم‌افزار متن‌باز (Open Source) یعنی کد منبع آن در دسترس است و طبق مجوزش می‌توانید آن را استفاده، مطالعه، تغییر و دوباره منتشر کنید. **نرم‌افزار آزاد** (Free Software) تقریباً همین است اما بر جنبه‌ی اخلاقی آزادی کاربر تأکید دارد. کلمه‌ی Free در اینجا به معنی «آزاد» است نه «رایگان»؛ جمله‌ی معروف Stallman: «Free as in freedom, not as in free beer».

چهار آزادی نرم‌افزار آزاد (FSF)

آزادی	معنی
۰	آزادی اجرای برنامه برای هر هدفی
۱	آزادی مطالعه‌ی طرز کار برنامه و تغییر آن (نیازمند دسترسی به کد منبع)
۲	آزادی توزیع نسخه‌های کپی برای کمک به دیگران
۳	آزادی توزیع نسخه‌های تغییریافته تا همه از بهبودها سود ببرند

FSF (بنیاد نرم‌افزار آزاد) این چهار آزادی را تعریف کرده و **OSI** (Open Source Initiative) تعریف «متن‌باز» و فهرست مجوزهای تأییدشده را نگه می‌دارد. اصطلاح **FOSS** یا **FLOSS** (Free/Libre and Open Source Software) هر دو جریان را با هم می‌گوید.

دو خانواده‌ی مجوز

نوع	ویژگی	مجوزهای مهم
Copyleft	نسخه‌ی تغییریافته و توزیع‌شده باید با همان مجوز و با کد منبع منتشر شود	GPLv2، GPLv3، AGPL؛ LGPL (کتابخانه‌ها، ضعیف‌تر)
Permissive	تقریباً هر کاری مجاز است، حتی استفاده در نرم‌افزار بسته؛ فقط نام سازنده حفظ شود	MIT، BSD، Apache 2.0
محتوا (نه کد)	برای متن، تصویر، ویدیو و آموزش	Creative Commons (مثل CC BY، CC BY-SA)

تشبیه

Copyleft مثل دستور غذای خانوادگی است که می‌گوید «هر کس این را تغییر داد و به دیگران داد، باید دستور جدید را هم آزاد بدهد». Permissive می‌گوید «هر کاری خواستی بکن، حتی در رستورانت بفروش، فقط اسم مادربزرگ را حذف نکن».

نکته‌ی آزمون

بدانید: کرنل لینوکس **GPLv2** است. **AGPL** حتی برای نرم‌افزاری که فقط به‌صورت سرویس تحت وب ارائه می‌شود، انتشار کد را الزامی می‌کند. **LGPL** اجازه می‌دهد نرم‌افزار بسته به کتابخانه‌ی LGPL لینک شود. Creative Commons برای کد استفاده نمی‌شود.

مدل‌های کسب‌وکار متن‌باز

متن‌باز بودن یعنی رایگان بودن نیست. شرکت‌ها از راه‌های مختلفی با نرم‌افزار متن‌باز درآمد دارند: فروش **پشتیبانی و اشتراک** (Red Hat، Canonical با Ubuntu Pro)، **آموزش و گواهی‌نامه** (LPI)، **مشاوره و سفارشی‌سازی**، **سرویس ابری مدیریت‌شده** و مدل **Dual licensing** (یک نسخه GPL و یک نسخه‌ی تجاری).

1.3 بررسی مجوز بسته‌ها در Ubuntu

Finding License Information

هر بسته‌ی Ubuntu فایلی به نام `copyright` در مسیر `/usr/share/doc/PACKAGE/` دارد که مجوز دقیق آن را بیان می‌کند. متن کامل مجوزهای رایج هم در `/usr/share/common-licenses` نصب است.

ls /usr/share/common-licenses

فهرست متن مجوزهای رایج نصب‌شده در سیستم.

```
$ ls /usr/share/common-licenses
```

less

خواندن صفحه‌به‌صفحه‌ی یک فایل؛ با q خارج شوید.

```
$ less /usr/share/doc/bash/copyright
```

grep -i license

پیدا کردن خط‌های شامل کلمه‌ی license بدون حساسیت به حروف بزرگ و کوچک.

```
$ grep -i license /usr/share/doc/bash/
copyright
```

head

چند خط اول فایل؛ با `-n` تعداد را تعیین کنید.

```
$ head -n 20 /usr/share/common-licenses/GPL-3
```

student@ubuntu-lab: ~

```
$ ls /usr/share/common-licenses
Apache-2.0  Artistic  BSD  CC0-1.0  GFDL  GFDL-1.2  GFDL-1.3  GPL  GPL-1  GPL-2  GPL-3
LGPL  LGPL-2  LGPL-2.1  LGPL-3  MPL-1.1  MPL-2.0
```

LAB 03

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

ممیزی مجوز بسته‌ها

مجوز چند بسته‌ی مهم سیستم را پیدا کنید و تفاوت Copyleft و Permissive را با مثال واقعی توضیح دهید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. فهرست مجوزهای مشترک نصب‌شده را ببینید و تعداد آن‌ها را بشمارید.

۲. فایل copyright بسته‌های `bash` ، `openssl` و `curl` را بررسی کنید.

۳. مشخص کنید کدام مجوز Copyleft و کدام Permissive است.

۴. برای یک شرکت که می‌خواهد کد را در محصول تجاری بسته استفاده کند، بین GPL و MIT کدام مناسب‌تر است؟ دلیل بنویسید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ ls /usr/share/common-licenses | wc -l  ## count lines
17
$ grep -m2 -i 'license' /usr/share/doc/bash/copyright
License: GPL-3+
$ grep -m2 -i 'license' /usr/share/doc/openssl/copyright
License: Apache-2.0
$ grep -m2 -i 'license' /usr/share/doc/curl/copyright
License: curl
```

✓ **بررسی نتیجه (Verify)**

bash با GPL-3 (Copyleft) و openssl نسخه‌ی ۳ با Apache-2.0 (Permissive) منتشر شده است. مجوز curl یک مجوز Permissive شبیه MIT است. مجوز دقیق به نسخه بستگی دارد؛ همیشه فایل copyright همان بسته را مرجع بگیرید.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. تفاوت کلیدی GPL و MIT چیست؟
2. برای انتشار یک کتاب آموزشی یا عکس از چه خانواده مجوزی استفاده می‌شود؟
3. آیا متن‌باز بودن به معنی رایگان بودن است؟
4. کرنل لینوکس با چه مجوزی منتشر می‌شود؟
5. سازمان OSI چه نقشی دارد؟

Objective 1.4

وزن در آزمون: ۲

مهارت‌های ICT، حریم خصوصی و SSH

ICT Skills and Working in Linux

کار با لینوکس فقط حفظ فرمان نیست. یک کاربر حرفه‌ای باید محیط‌های کاری مختلف را بشناسد، در اینترنت امن رفتار کند، از ابزارهای ابری درست استفاده کند و بتواند به سرورهای راه دور وصل شود.

محیط‌های میزکار (Desktop Environment)

لینوکس برخلاف Windows یک محیط گرافیکی ثابت ندارد. **GNOME** (پیش‌فرض Ubuntu و Fedora)، **KDE Plasma** (پرامکانات و قابل‌تنظیم)، **Xfce** و **LXQt** (سبک برای سیستم‌های ضعیف) و **Cinnamon** و **MATE** (در Linux Mint) مشهورترین‌اند. زیر همه‌ی این‌ها یک **Display Server** قرار دارد: **X11** (قدیمی) یا **Wayland** (جدید و پیش‌فرض Ubuntu 24.04). سرورها معمولاً بدون محیط گرافیکی و فقط با خط فرمان کار می‌کنند.

حریم خصوصی و امنیت در اینترنت

Cookie	فایل کوچکی که سایت در مرورگر ذخیره می‌کند؛ کوکی‌های شخص ثالث برای ردیابی استفاده می‌شوند.
Private browsing	تاریخچه و کوکی را پس از بستن پنجره پاک می‌کند، اما IP و فعالیت شما را از سایت و ISP پنهان نمی‌کند.
HTTPS	نسخه‌ی رمزنگاری‌شده‌ی HTTP با TLS؛ قفل کنار آدرس یعنی ارتباط رمز شده است.
Password manager	برنامه‌ای مثل KeePassXC که رمزهای قوی و جدا برای هر سایت نگه می‌دارد.
2FA/MFA	ورود دومرحله‌ای؛ رمز به‌علاوه‌ی یک کد یا کلید سخت‌افزاری.
GnuPG	ابزار متن‌باز رمزنگاری و امضای ایمیل و فایل؛ پیاده‌سازی استاندارد OpenPGP.
Disk encryption	رمزنگاری کل دیسک با LUKS که هنگام نصب Ubuntu قابل انتخاب است.
Cloud computing	استفاده از سرویس‌های راه دور: SaaS (مثل ایمیل وب)، PaaS و IaaS (ماشین مجازی ابری).

نکته‌ی آزمون

در آزمون به **Private browsing** حساس‌اند: این حالت فقط ردپا را روی **همان کامپیوتر** پاک می‌کند. همچنین بدانید رمز قوی طولانی و منحصربه‌فرد است و استفاده‌ی یک رمز برای چند سایت خطر بزرگی است.

ssh اتصال امن راه دور با SSH

Secure Shell

SSH (Secure Shell) پروتکل استاندارد برای ورود امن به یک ماشین لینوکس از راه دور است. همه‌ی ترافیک، از جمله رمز عبور، رمزنگاری می‌شود. SSH جایگزین امن پروتکل‌های قدیمی **Telnet** و **rlogin** است که رمز را به‌صورت متن ساده می‌فرستادند. پورت پیش‌فرض SSH عدد **۲۲** است. سمت سرور برنامه‌ی `sshd` (بسته‌ی openssh-server) و سمت کاربر برنامه‌ی `ssh` (بسته‌ی openssh-client) قرار دارد.

ساختار کلی · SYNTAX

```
ssh [OPTIONS] [user@]host [command]
```

سوییچ‌های رایج `ssh`

سوییچ	شکل بلند	کاربرد	مثال
`-p PORT`		اتصال به پورتی غیر از ۲۲	`ssh -p 2222 student@10.0.0.5`
`-i FILE`		استفاده از کلید خصوصی مشخص	`ssh -i ~/.ssh/lab_key student@host`
`-v`		حالت پرحرف برای عیب‌یابی اتصال	`ssh -v student@host`
`-X`		انتقال پنجره‌ی گرافیکی (X11 forwarding)	`ssh -X student@host`
`-L`		تونل پورت محلی	`ssh -L 8080:localhost:80 host`

ssh-keygen

ساخت جفت کلید عمومی و خصوصی؛ نوع ed25519 امن و کوتاه است.

```
$ ssh-keygen -t ed25519
```

ssh-copy-id

کپی کلید عمومی روی سرور برای ورود بدون رمز.

```
$ ssh-copy-id student@192.168.56.10
```

scp

کپی امن فایل از روی SSH؛ با `-r` برای پوشه.

```
$ scp notes.txt student@host:/tmp/
```

systemctl status ssh

وضعیت سرویس SSH روی سرور.

```
$ systemctl status ssh
```

هشدار

کلید خصوصی (`~/.ssh/id_ed25519`) را هرگز با کسی به اشتراک نگذارید، در ایمیل نفرستید و در عکس صفحه منتشر نکنید. فقط کلید عمومی (فایل `.pub`) روی سرورها کپی می‌شود.

LAB 04

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

اولین اتصال SSH

سرویس SSH را نصب و فعال کنید، به localhost وصل شوید، یک کلید ed25519 بسازید و ورود با کلید را تجربه کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱ مطمئن شوید بسته‌ی `openssh-server` نصب و سرویس `ssh` فعال است.

۲ به همان ماشین (`localhost`) با کاربر `student` و رمز عبور وصل شوید؛ fingerprint را بپذیرید و خارج شوید.

۳ یک جفت کلید `ed25519` بسازید.

۴ کلید عمومی را با `ssh-copy-id` روی localhost کپی کنید و دوباره وصل شوید؛ این بار رمز پرسیده نمی‌شود (یا فقط passphrase کلید).

۵ یک فرمان را از راه دور اجرا کنید بدون اینکه وارد شِل شوید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ sudo apt install -y openssh-server
$ systemctl is-active ssh
active
$ ssh student@localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:Xq3...
Are you sure you want to continue connecting (yes/no)? yes
student@localhost's password:
student@ubuntu-lab:~$ exit
$ ssh-keygen -t ed25519 -C "student@ubuntu-lab"  ## press Enter for default path
Your public key has been saved in /home/student/.ssh/id_ed25519.pub
$ ssh-copy-id student@localhost
$ ssh student@localhost 'hostname; uptime'
ubuntu-lab
 12:10:03 up 1:02,  2 users,  load average: 0.01, 0.05, 0.03
```

VM

VMware Workstation

اگر بخواهید از Windows (Host) به VM وصل شوید، IP کارت Host-Only را با `ip -br addr` پیدا کنید و در PowerShell بنویسید `ssh student@192.168.56.128` (Windows 10 و 11 کلاینت SSH داخلی دارند).

✓ **بررسی نتیجه (Verify)**

در اولین اتصال باید پرسش fingerprint را ببینید؛ پس از `ssh-copy-id` ، ورود دوم بدون رمز کاربر انجام می‌شود. فایل `~/.ssh/authorized_keys` باید شامل کلید عمومی شما باشد.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. چرا SSH و HTTPS از Telnet و HTTP امن‌ترند؟
2. Private browsing چه چیزی را پنهان نمی‌کند؟
3. پورت پیش‌فرض SSH چیست؟
4. کدام فایل روی سرور کلیدهای عمومی مجاز را نگه می‌دارد؟
5. محیط گرافیکی پیش‌فرض Ubuntu 24.04 چیست و از کدام Display Server استفاده می‌کند؟

EXAM 010-160 · TOPIC 2

2

پیدا کردن راه در سیستم لینوکس

FINDING YOUR WAY ON A LINUX SYSTEM

از prompt و shell تا متغیرها، راهنما، مسیرها، فهرست فایل‌ها، ساخت و کپی و حذف و جستجوی فایل با find؛ مهم‌ترین فصل برای شروع کار عملی

وزن در آزمون: ۹ سؤال از ۴۰

2.1 مبانی خط فرمان و Bash

2.2 گرفتن کمک از خط فرمان

2.3 دایرکتوری‌ها و فهرست فایل‌ها

2.4 ساخت، کپی، جابه‌جایی و حذف

2.5 جستجوی فایل با find و locate

Objective 2.1

وزن در آزمون: ۳

مبانی خط فرمان و Bash

Command Line Basics

وقتی ترمینال را باز می‌کنید، برنامه‌ای به نام **Shell** (پوسته) اجرا می‌شود. شِل مترجمی است بین شما و سیستم‌عامل: متنی را که تایپ می‌کنید می‌خواند، آن را به **فرمان، گزینه‌ها و آرگومان‌ها** تقسیم می‌کند، برنامه‌ی مربوط را پیدا و اجرا می‌کند و نتیجه را نشان می‌دهد. شِل پیش‌فرض Ubuntu و بیشتر توزیع‌ها **Bash** (Bourne Again SHell) است. شِل‌های دیگری هم هستند: `sh` (شِل کلاسیک)، `zsh` (پیش‌فرض macOS)، `fish`، `ksh` و `csh`.

تشبیه

ترمینال مثل یک پنجره‌ی پیشخوان بانک است و شِل کارمند پشت آن. شما درخواست را به زبان خاص او می‌نویسید (فرمان)، کارمند آن را می‌خواند، به بخش مربوط می‌فرستد و نتیجه را از همان پنجره به شما برمی‌گرداند.

آناتومی Prompt

```
student@ubuntu-lab: ~
student@ubuntu-lab:~$
```

بخش	معنی
student	نام کاربری که وارد شده است
@ubuntu-lab	نام میزبان (hostname) ماشین
:~	دایرکتوری فعلی؛ علامت `~` یعنی خانه‌ی کاربر (`/home/student`)
$	کاربر عادی هستید. اگر `#` ببینید یعنی root هستید و باید بیشتر احتیاط کنید

ساختار یک فرمان

ساختار کلی · SYNTAX

```
command [options] [arguments]
```

Command	نام برنامه یا فرمانی که می‌خواهید اجرا شود، مثل `ls`.
Option (Switch)	رفتار فرمان را تغییر می‌دهد. کوتاه با یک خط تیره (`-l`) یا بلند با دو خط تیره (`--all`).
Argument	چیزی که فرمان روی آن کار می‌کند؛ معمولاً نام فایل یا مسیر، مثل `/etc`.

```
student@ubuntu-lab: ~
$ ls -l -a /etc  ## command + 2 short options + 1 argument
$ ls -la /etc  ## short options can be combined
$ ls --all --human-readable /etc  ## long options are self-describing
$ ls -l --sort=time /var/log  ## long option with a value
```

هشدار

لینوکس به **حروف بزرگ و کوچک حساس** (case-sensitive) است. `ls` و `LS` دو چیز متفاوت‌اند و `LS` اصلاً وجود ندارد. فایل‌های `Report.txt` و `report.txt` هم دو فایل جدا هستند. همچنین فاصله جداکننده است: `ls-l` یک فرمان ناشناخته است، نه `ls -l` .

Bash

انواع فرمان: Builtin، External، Alias

Command Types

شِل فرمان‌ها را از چند جا پیدا می‌کند. دانستن نوع فرمان به شما می‌گوید برای راهنما کجا بروید و چرا یک فرمان رفتار خاصی دارد.

نوع	توضیح	مثال
Builtin	داخل خود Bash ساخته شده؛ فایل جداگانه ندارد	`cd` ، `echo` ، `export` ، `type` ، `history` ، `alias`
External	یک فایل اجرایی روی دیسک که از مسیرهای PATH پیدا می‌شود	`ls` در `/usr/bin/ls` ، `grep` ، `find`
Alias	نام مستعار که شِل قبل از اجرا جایگزین می‌کند	`ll` که در Ubuntu معادل `ls -alF` است
Function	تابعی که در شِل تعریف شده	توابع تعریف‌شده در `~/.bashrc`
Keyword	کلمه‌ی کلیدی زبان شِل	`if` ، `for` ، `while` ، `[[`

type

نوع فرمان را می‌گوید (builtin، alias، file). با `-a` همه‌ی مکان‌ها را نشان می‌دهد.

```
$ type cd ls ll
```

which

مسیر فایل اجرایی external را از PATH پیدا می‌کند.

```
$ which grep
```

whereis

فایل اجرایی، سورس و صفحه‌ی man را پیدا می‌کند.

```
$ whereis ls
```

alias

ساخت یا نمایش نام مستعار؛ `unalias` آن را حذف می‌کند.

```
$ alias c='clear'
```

student@ubuntu-lab: ~

```
$ type cd ls ll
cd is a shell builtin
ls is aliased to `ls --color=auto'
ll is aliased to `ls -alF'
$ which ls
/usr/bin/ls
$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
$ alias rm='rm -i'  ## ask before each delete (only in this shell)
$ unalias rm
```

نکته

aliasی که در ترمینال می‌سازید با بستن ترمینال از بین می‌رود. برای دائمی‌کردن، آن را در انتهای فایل `~/.bashrc` بنویسید و با `source ~/.bashrc` دوباره بارگذاری کنید.

متغیرها، متغیرهای محیطی و PATH

$VAR

Shell and Environment Variables

متغیر یک نام است که مقداری را نگه می‌دارد. با `NAME=value` تعریف می‌شود و با `$NAME` مقدارش خوانده می‌شود. نکته‌ی مهم: **دو طرف علامت = هیچ فاصله‌ای نباید باشد.**

student@ubuntu-lab: ~

```
$ COURSE=Linux  ## correct
$ echo $COURSE
Linux
$ COURSE = Linux  ## WRONG: bash thinks COURSE is a command
COURSE: command not found
$ echo "I am learning $COURSE on $HOSTNAME"
I am learning Linux on ubuntu-lab
```

متغیر محلی و متغیر محیطی

متغیری که فقط با `NAME=value` ساخته شود **محلی** (Shell variable) است و برنامه‌هایی که از این شِل اجرا می‌کنید آن را نمی‌بینند. با `export` آن را به **متغیر محیطی** (Environment variable) تبدیل می‌کنیم تا به همه‌ی فرایندهای فرزند منتقل شود.

متغیرهای محیطی مهم

متغیر	مقدار نمونه	معنی
HOME	/home/student	خانه‌ی کاربر؛ همان `~`
USER	student	نام کاربر فعلی
SHELL	/bin/bash	شِل پیش‌فرض کاربر
PWD	/home/student/lab	دایرکتوری فعلی
PATH	/usr/local/bin:/usr/bin:/bin:...	مسیرهایی که شِل برای پیدا کردن فرمان می‌گردد
PS1	u@\h:\w\$\	قالب prompt
LANG	en_US.UTF-8	زبان و encoding
HISTSIZE	1000	تعداد فرمان‌های نگه‌داشته‌شده در history

echo $VAR

نمایش مقدار یک متغیر.

```
$ echo $PATH
```

export

تبدیل به متغیر محیطی؛ `export NAME=value` هم مجاز است.

```
$ export EDITOR=nano
```

env

فهرست همه‌ی متغیرهای محیطی.

```
$ env | less
```

set

همه‌ی متغیرها و توابع شِل (محلی و محیطی).

```
$ set | head
```

unset

حذف یک متغیر.

```
$ unset COURSE
```

printenv

نمایش یک یا همه‌ی متغیرهای محیطی.

```
$ printenv HOME
```

؟ نکته‌ی آزمون

PATH فهرستی از دایرکتوری‌هاست که با `:` از هم جدا شده‌اند. وقتی `ls` را تایپ می‌کنید، شِل این دایرکتوری‌ها را به ترتیب می‌گردد. برای اجرای برنامه‌ای که در PATH نیست باید مسیرش را بدهید، مثل `./script.sh` . افزودن مسیر به PATH: `export PATH="$PATH:$HOME/bin"` .

Quote

نقل‌قول و Escape

Quoting

بعضی نویسه‌ها برای شِل معنای خاص دارند: فاصله (جداکننده)، `$` (متغیر)، `*` (Globbing)، `;` و `|` و `&` و `>`. برای اینکه شِل آن‌ها را عادی ببیند، از نقل‌قول استفاده می‌کنیم.

روش	رفتار	مثال	خروجی
"دابل‌کوتیشن"	متغیرها و `$( )` گسترش می‌یابند؛ فاصله حفظ می‌شود	`echo "Home: $HOME"`	Home: /home/student
'سینگل‌کوتیشن'	همه‌چیز عیناً و بدون گسترش	`echo 'Home: $HOME'`	Home: $HOME
بک‌اسلش \	فقط نویسه‌ی بعدی را عادی می‌کند	`echo \$HOME`	$HOME
$( )	جایگزینی فرمان؛ خروجی فرمان جایگزین می‌شود	`echo "Today: $(date +%F)"`	Today: 2026-10-04

student@ubuntu-lab: ~

```
$ NAME='Mohammad Reza'
$ echo "Hello $NAME"
Hello Mohammad Reza
$ echo 'Hello $NAME'
Hello $NAME
$ mkdir "My Project"  ## one directory with a space in its name
$ mkdir My Project  ## two directories: My and Project
$ echo "Kernel: $(uname -r)"
Kernel: 6.8.0-45-generic
```

فرمان echo و سوییچ‌هایش

سوییچ	شکل بلند	کاربرد	مثال
`-n`		بدون رفتن به خط بعد در پایان	`echo -n "Name: "`
`-e`		تفسیر نویسه‌های خاص مثل \n (خط جدید) و \t (تب)	`echo -e "a\tb\nc"`
`-E`		عدم تفسیر نویسه‌های خاص (پیش‌فرض)	`echo -E "a\nb"`

نکته

برای نام فایل‌ها از فاصله و نویسه‌های فارسی استفاده نکنید. نام‌هایی مثل `my_project` یا `my-project` کار در خط فرمان و اسکریپت را بسیار ساده‌تر می‌کنند.

history تاریخچه، میانبرها و ترکیب فرمان‌ها

History, Shortcuts and Command Lists

Bash هر فرمانی را که اجرا می‌کنید در حافظه و در پایان نشست در فایل `~/.bash_history` ذخیره می‌کند.

فرمان `history` و گسترش تاریخچه

سوییچ	شکل بلند	کاربرد	مثال
`history`		نمایش فهرست شماره‌دار فرمان‌های قبلی	`history`
`history N`		نمایش N فرمان آخر	`history 10`
`history -c`		پاک‌کردن تاریخچه‌ی نشست فعلی	`history -c`
`!!`		اجرای دوباره‌ی فرمان قبلی	`sudo !!`
`!N`		اجرای فرمان شماره‌ی N از تاریخچه	`!42`
`!string`		اجرای آخرین فرمانی که با string شروع شده	`!ssh`
`!$`		آخرین آرگومان فرمان قبلی	`cd !$`

کلیدهای ویرایش خط فرمان (Readline)

کلید	کار
Tab	تکمیل خودکار نام فرمان، فایل و مسیر؛ دو بار Tab همه‌ی گزینه‌ها را نشان می‌دهد
↑ و ↓	پیمایش در فرمان‌های قبلی
Ctrl+R	جستجوی معکوس در تاریخچه؛ چند حرف تایپ کنید
Ctrl+A و Ctrl+E	رفتن به ابتدا و انتهای خط
Ctrl+U و Ctrl+K	پاک‌کردن از مکان‌نما تا ابتدا یا تا انتهای خط
Ctrl+W	پاک‌کردن کلمه‌ی قبلی
Ctrl+L	پاک‌کردن صفحه (مثل clear)
Ctrl+C	توقف فرمان در حال اجرا
Ctrl+D	پایان ورودی؛ در prompt خالی، خروج از شِل

ترکیب فرمان‌ها و کد خروج

هر فرمان پس از پایان یک **کد خروج** (Exit status) برمی‌گرداند: **صفر یعنی موفقیت** و هر عدد دیگری یعنی خطا. این مقدار در متغیر ویژه‌ی `$?` ذخیره می‌شود.

عملگر	معنی	مثال
;	اجرای پشت‌سرهم، بدون توجه به نتیجه	`cd /tmp; ls`
&&	دومی فقط اگر اولی موفق بود	`mkdir lab && cd lab`
II (دو خط عمودی)	دومی فقط اگر اولی ناموفق بود	`ping -c1 host II echo down`

student@ubuntu-lab: ~

```
$ ls /etc/hostname; echo $?
/etc/hostname
0
$ ls /nothing; echo $?
ls: cannot access '/nothing': No such file or directory
2
$ mkdir -p ~/lab && cd ~/lab && pwd
/home/student/lab
```

دو خط عمودی

در جدول بالا به‌خاطر محدودیت قالب، عملگر «یا» را II نوشته‌ایم؛ در ترمینال آن را با دو بار فشردن کلید | (Shift+\) تایپ کنید: || .

LAB 05

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

شناخت Shell، متغیر، نقل‌قول و تاریخچه

تفاوت متغیر محلی و exportشده را در یک شِل فرزند ببینید، نوع فرمان‌ها را تشخیص دهید، با نقل‌قول کار کنید و از تاریخچه استفاده کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. شِل فعلی و نسخه‌ی Bash را پیدا کنید (`echo $SHELL` و `bash --version`).

۲. متغیر `NAME` را با مقدار نام خودتان (با فاصله) بسازید و با دو نوع نقل‌قول چاپ کنید.

۳. بررسی کنید که یک شِل فرزند (`bash -c`) متغیر را می‌بیند یا نه؛ سپس آن را export کنید و دوباره بررسی کنید.

۴. نوع فرمان‌های `cd` ، `ls` ، `ll` ، `echo` و `grep` را مشخص کنید.

۵. مسیر جدید `~/bin` را به PATH اضافه کنید و نتیجه را ببینید.

۶. با `history` و `!` یک فرمان قبلی را دوباره اجرا کنید و کد خروج یک فرمان موفق و ناموفق را ببینید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ echo $SHELL; bash --version | head -1
/bin/bash
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
$ NAME='Mohammad Reza'
$ echo "Hello $NAME"; echo 'Hello $NAME'
Hello Mohammad Reza
Hello $NAME
$ bash -c 'echo "child sees: [$NAME]"'
child sees: []
$ export NAME
$ bash -c 'echo "child sees: [$NAME]"'
child sees: [Mohammad Reza]
$ type cd ls ll echo grep
$ mkdir -p ~/bin && export PATH="$PATH:$HOME/bin"
$ echo $PATH | tr ':' '\n' | tail -2
/snap/bin
/home/student/bin
$ history | tail -3
$ !echo
$ ls /nothing; echo "exit code: $?"
exit code: 2
```

✓ بررسی نتیجه (Verify)

قبل از `export` ، شِل فرزند مقدار خالی چاپ می‌کند و بعد از آن نام شما را. `type` باید نشان دهد `cd` و `echo` builtin، `ls` و `ll` alias و `grep` نیز alias یا فایل `/usr/bin/grep` است.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. در prompt علامت `#` به‌جای `$` چه معنایی دارد؟
2. چرا `NAME = Reza` خطا می‌دهد؟
3. متغیر `$?` چه چیزی را نگه می‌دارد؟
4. فرق سینگل‌کوتیشن و دابل‌کوتیشن چیست؟
5. متغیر PATH چه کاربردی دارد؟
6. فرمان `mkdir lab && cd lab` چه تفاوتی با `mkdir lab; cd lab` دارد؟

Objective 2.2

وزن در آزمون: ۲

گرفتن کمک از خط فرمان

Using the Command Line to Get Help

هیچ مدیر سیستمی همه‌ی گزینه‌های همه‌ی فرمان‌ها را حفظ نیست. مهارت حرفه‌ای این است که **بدانید جواب را کجا پیدا کنید**؛ آن هم بدون اینترنت و از داخل خود سیستم. لینوکس چند سیستم راهنمای داخلی دارد:

ابزار	کاربرد	مثال
`--help`	خلاصه‌ی سریع گزینه‌ها؛ تقریباً همه‌ی فرمان‌های external دارند	`ls --help`
`man`	صفحه‌ی راهنمای کامل (Manual)	`man ls`
`help`	راهنمای فرمان‌های builtin خود Bash	`help cd`
`info`	مستندات ساختاریافته و پیوندی پروژه‌ی GNU	`info coreutils`
`whatis`	توضیح یک‌خطی از صفحه‌ی man	`whatis passwd`
`apropos`	جستجوی کلمه در عنوان و توضیح همه‌ی صفحات man	`apropos copy`
`/usr/share/doc`	مستندات، README و changelog بسته‌ها	`ls /usr/share/doc/bash`

هشدار

`cd` یک builtin است و فایل جداگانه ندارد؛ پس `cd --help` و `man cd` ممکن است کار نکنند یا نتیجه‌ی کلی بدهند. برای builtinها همیشه از `help` استفاده کنید: `help cd` ، `help export` ، `help history` .

man صفحات راهنمای man

The Manual Pages

SYNTAX · ساختار کلی
```
man [SECTION] PAGE
```

صفحات man به **بخش‌های شماره‌دار** تقسیم شده‌اند. یک نام ممکن است در چند بخش وجود داشته باشد؛ مثلاً `passwd` هم یک فرمان است (بخش ۱) و هم یک فایل (`/etc/passwd`، بخش ۵).

بخش‌های man

بخش	محتوا	مثال
1	فرمان‌های کاربر	`man 1 ls`
2	فراخوانی‌های سیستمی کرنل	`man 2 open`
3	توابع کتابخانه‌ای (C)	`man 3 printf`
4	فایل‌های ویژه و دستگاه‌ها (/dev)	`man 4 null`
5	قالب فایل‌های تنظیمات	`man 5 passwd`
6	بازی‌ها	`man 6 fortune`
7	مفاهیم و قراردادها	`man 7 regex`
8	فرمان‌های مدیریتی سیستم (root)	`man 8 useradd`

ساختار یک صفحه‌ی man

هر صفحه بخش‌های استانداردی دارد: **NAME** (نام و توضیح یک‌خطی)، **SYNOPSIS** (ساختار فرمان؛ چیزهای داخل `[ ]` اختیاری‌اند و `...` یعنی قابل تکرار)، **DESCRIPTION** (توضیح و گزینه‌ها)، **EXAMPLES** (در برخی صفحات)، **FILES**، **SEE ALSO** (صفحات مرتبط) و **AUTHOR**.

کلیدهای حرکت داخل man (و less)

کلید	کار
Space یا PgDn	صفحه‌ی بعد
b یا PgUp	صفحه‌ی قبل
↑ ↓ یا j k	یک خط بالا و پایین
g و G	ابتدا و انتهای صفحه
/word	جستجوی رو به جلو برای word

کلید	کار
n و N	نتیجه‌ی بعدی و قبلی جستجو
h	راهنمای کلیدها
q	خروج

سوییچ‌های رایج `man`

سوییچ	شکل بلند	کاربرد	مثال
`-k`	`--apropos`	جستجوی کلمه در عنوان صفحات (مثل apropos)	`man -k password`
`-f`	`--whatis`	توضیح یک‌خطی (مثل whatis)	`man -f passwd`
`-a`	`--all`	نمایش پشت‌سرهم همه‌ی بخش‌های یک نام	`man -a passwd`
`-w`	`--where`	مسیر فایل صفحه‌ی man	`man -w ls`

```
student@ubuntu-lab: ~
$ whatis passwd
passwd (1ssl)        - OpenSSL application commands
passwd (1)           - change user password
passwd (5)           - the password file
$ apropos "copy files"
cp (1)               - copy files and directories
$ man -w ls
/usr/share/man/man1/ls.1.gz
```

نکته

اگر `apropos` یا `man -k` چیزی پیدا نکرد، پایگاه داده‌ی صفحات man را با `sudo mandb` به‌روز کنید.

info ،help ،--help و مستندات بسته‌ها

Other Help Sources

help

راهنمای builtinهای Bash؛ `help` به‌تنهایی فهرست همه‌ی builtinها را می‌دهد.

```
$ help cd
```

--help

خلاصه‌ی گزینه‌های یک فرمان external؛ سریع‌ترین راه.

```
$ cp --help | less
```

info

مستندات GNU با گره‌ها و پیوندها؛ کلیدهای n و p و u و q.

```
$ info ls
```

/usr/share/doc

README، changelog و مثال‌های هر بسته.

```
$ ls /usr/share/doc/openssh-server
```

student@ubuntu-lab: ~

```
$ help cd | head -3
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.
$ ls --help | grep -- '-h,'
  -h, --human-readable   with -l and -s, print sizes like 1K 234M 2G etc.
```

کلیدهای info

کلید	کار
n و p	گره (صفحه‌ی) بعدی و قبلی
u	بالا رفتن به گره‌ی والد
Enter روی پیوند	رفتن به پیوند
l	برگشت به صفحه‌ی قبلی دیده‌شده
q	خروج

نکته‌ی آزمون

ترتیب منطقی برای گرفتن کمک: ۱) `--help` برای یادآوری سریع، ۲) `man` برای جزئیات، ۳) `info` برای مستندات GNU، ۴) `/usr/share/doc` . در آزمون می‌پرسند صفحه‌ی man قالب فایل `/etc/passwd` در کدام بخش است (بخش ۵) و فرمان‌های مدیریتی در کدام بخش (بخش ۸).

LAB 06

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

کار با راهنمای سیستم

صفحه‌ی man مناسب را پیدا کنید، داخل آن جستجو کنید، بخش‌های مختلف را تشخیص دهید و برای builtinها از help استفاده کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱ صفحه‌ی man فرمان `ls` را باز کنید؛ داخل آن کلمه‌ی `sort` را جستجو کنید، با `n` نتیجه‌ی بعدی را ببینید و با `q` خارج شوید.

۲ با `whatis` ببینید `passwd` در کدام بخش‌ها صفحه دارد. صفحه‌ی بخش ۵ را باز کنید.

۳ با `apropos` فرمانی برای «تغییر مالک فایل» (owner) پیدا کنید.

۴ راهنمای `cd` را با `help` ببینید و توضیح دهید چرا `man cd` مناسب نیست.

۵ فقط خط مربوط به گزینه‌ی `-r` فرمان `cp` را از خروجی `--help` بیرون بکشید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ man ls  ## inside man: /sort  then n  then q
$ whatis passwd
passwd (1)           - change user password
passwd (5)           - the password file
$ man 5 passwd
$ apropos owner | grep -w chown
chown (1)            - change file owner and group
$ help cd | head -2
$ type cd
cd is a shell builtin
$ cp --help | grep -- ' -r'
  -R, -r, --recursive          copy directories recursively
```

نکته

دو خط تیره `--` در `grep -- ' -r'` به grep می‌گوید «گزینه‌ها تمام شد»؛ وگرنه grep الگوی `-r` را گزینه‌ی خودش فرض می‌کند. این قرارداد در بیشتر فرمان‌ها کار می‌کند؛ مثلاً حذف فایلی به نام `-file` : `rm -- -file` .

بررسی نتیجه (Verify)

باید بتوانید بدون اینترنت و فقط با man و help به هر سؤال درباره‌ی گزینه‌ها جواب دهید. خروجی `whatis passwd` حداقل دو بخش ۱ و ۵ را نشان می‌دهد.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. برای دیدن راهنمای `cd` از چه فرمانی استفاده می‌کنیم و چرا؟
2. در `man 5 passwd` چه چیزی می‌بینیم؟
3. چطور داخل یک صفحه‌ی man کلمه‌ای را جستجو می‌کنیم؟
4. اگر نام فرمانی را نمی‌دانید اما کارش را می‌دانید، از چه فرمانی استفاده می‌کنید؟
5. فرمان‌های مدیریتی مثل useradd در کدام بخش man قرار دارند؟

Objective 2.3

وزن در آزمون: ۲

دایرکتوری‌ها و فهرست فایل‌ها

Using Directories and Listing Files

لینوکس یک درخت فایل هندسی از ریشه‌ی `/` دارد؛ درایوهای C و D و دسکتاپ وجود ندارند. هرچیز فایل است: فایل معمولی، دایرکتوری، دستگاه، سوکت. **مسیر مطلق** از `/` شروع می‌شود و **مسیر نسبی** از دایرکتوری فعلی.

Root /	ریشه‌ی درخت فایل.
Current directory .	دایرکتوری فعلی.
Parent directory ..	دایرکتوری والد (یک سطح بالاتر).
Home directory ~	خانه‌ی کاربر فعلی (معادل $HOME).
Absolute path	مسیری که از / شروع می‌شود، مثل `/home/student/lab` .
Relative path	مسیری نسبت به دایرکتوری فعلی، مثل `lab/docs` یا `../../etc` .
Hidden file	فایلی که نام آن با نقطه شروع می‌شود (مثل `.bashrc`).

pwd

چاپ دایرکتوری کاری فعلی (Working Directory).

```
$ pwd
```

cd DIRECTORY

تغییر دایرکتوری؛ بدون آرگومان به خانه و با `-` به محل قبلی برمی‌رود.

```
$ cd /var/log
```

cd ..

رفتن به دایرکتوری والد.

```
$ cd ..
```

cd ~ یا cd

رفتن به خانه.

```
$ cd
```

ls

فهرست‌گرفتن دایرکتوری فعلی یا دایرکتوری داده‌شده.

```
$ ls -la /etc
```

tree

نمایش درختی سازمان پوشه‌ها (نیاز به بسته‌ی tree).

```
$ tree -L 2 /usr
```

سوییچ‌های پرکاربرد `ls`

سوییچ	شکل بلند	کاربرد	مثال
`-l`		خط فرمت (long): مجوز، مالک، اندازه، تاریخ، نام	`ls -l`

سوییچ	شکل بلند	کاربرد	مثال
`-a`	`--all`	نمایش فایل‌های مخفی (شروع با نقطه)	`ls -a`
`-h`	`--human-readable`	اندازه‌ها به فرمت خوانا (K، M، G)	`ls -lh`
`-S`		مرتب‌سازی بر اساس اندازه (بزرگ‌ترین اول)	`ls -S`
`-t`		مرتب‌سازی بر اساس زمان تغییر (تازه‌ترین اول)	`ls -lt`
`-r`		معکوس کردن ترتیب	`ls -ltr`
`-d`		فقط خود دایرکتوری، نه محتویات	`ls -ld /etc`
`-R`		نمایش بازگشتی؛ همه‌ی دایرکتوری‌های فرعی	`ls -R /etc`

student@ubuntu-lab: ~

```
$ pwd
/home/student
$ cd /var/log; pwd; ls | head -4
/var/log
auth.log
boot.log
dmesg
$ ls -lah ~ | head -3
drwxr-xr-x 8 student student 4.0K Oct  4 10:30 .
-rw-r--r-- 1 student student  220 Nov 24  2022 .bash_logout
-rw-r--r-- 1 student student 3.7K Nov 24  2022 .bashrc
$ ls -ld /etc
drwxr-xr-x 1 root root 4096 Oct  4 08:15 /etc
```

هشدار

فرمان `ls -l` و `ls -ld` تفاوت دارند: اول محتویات `/etc` را فهرست می‌کند، دوم فقط مشخصات خود `/etc` را نشان می‌دهد.

نکته‌ی آزمون

بدانید `ls -a` مخفی‌ها را نشان می‌دهد و `ls -h` اندازه‌ها را خوانا می‌کند. ترکیب `ls -lah` بسیار رایج است.

find جستجوی فایل با find

The find Command

`find` یک ابزار قدرتمند برای جستجوی فایل‌های پیچیده است. روی درخت فایل پیمایش می‌کند و بر اساس معیارهایی مثل نام، نوع، اندازه و زمان فایل‌ها را پیدا می‌کند.

SYNTAX · ساختار کلی

```
find [PATH] [CONDITION]...
```

شرایط رایج `find`

سوییچ	شکل بلند	کاربرد	مثال
`-name PATTERN`		جستجو بر اساس نام فایل (Glob pattern)	`find /etc -name '*.conf'`
`-type TYPE`		نوع فایل: f (معمولی)، d (دایرکتوری)، l (لینک)	`find / -type f -name '*.log'`
`-size SIZE`		اندازه فایل؛ `+` (بیشتر از)، `-` (کمتر از)	`find ~ -type f -size +10M`
`-mtime N`		تغییر در N روز قبل؛ `-7` (آخر ۷ روز)	`find /tmp -type f -mtime -7`
`-user USERNAME`		مالک فایل	`find / -user student`
`-group GROUPNAME`		گروه مالک	`find / -group sudo`
`-perm MODE`		مجوز دقیق (بحث در فصل ۵)	`find / -perm -4000`

سوییچ	شکل بلند	کاربرد	مثال
`-exec COMMAND`		اجرای فرمان روی هر نتیجه؛ `{}` جای نام فایل و `;` انجام	`find . -name '*.tmp' -exec rm {} \;`
`-delete`		حذف فایل‌های یافت‌شده (احتیاط!)	`find . -name '*.bak' -delete`
`-prune`		جاسازی دایرکتوری خاصی	`find / -name node_modules -prune -o -name '*.js' -print`

student@ubuntu-lab: ~

```
$ find ~/ -type f -name '*.txt' | head -3
/home/student/lab/docs/note.txt
$ find /tmp -type f -mtime -1  ## modified in last 1 day
$ find /etc -type f -size 0  ## empty files
$ find /usr -name 'python*' -type f
```

هشدار

`find` روی تمام درخت جستجو می‌کند و معمولاً زمان‌برتر است. برای جستجوی سریع در نام‌های فایل اول‌شده، `locate` بسیار سریع‌تر است (اما روی دیتابیس مبتنی است و تازه‌ترین تغییرات را ندارد).

locate جستجوی سریع با locate

Finding Files with locate

`locate` از یک دیتابیس (معمولاً روزشده شب) جستجو می‌کند و بسیار سریع است. در Ubuntu بسته‌ی `plocate` نسخه‌ی مدرن و سریع است.

locate FILE

جستجو در دیتابیس نام‌های فایل

```
$ locate sshd_config
```

locate -c

تعداد نتایج را می‌شمارد

```
$ locate -c ssh
```

locate -i

جستجوی غیرحساس به بزرگی حروف

```
$ locate -i BASH
```

updatedb

به‌روزرسانی دیتابیس (معمولاً هر شب خودکار)

```
$ sudo updatedb
```

student@ubuntu-lab: ~

```
$ locate sshd_config
/etc/ssh/sshd_config
/etc/ssh/sshd_config.d
$ locate -c firefox
4
```

نکته

اگر فایل تازه‌ای را ساختید و locate آن را نیافت، دیتابیس را با `sudo updatedb` تازه کنید و دوباره سعی کنید.

LAB 07

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

پیمایش درخت فایل و مقایسه‌ی ls با find

مسیرهای مطلق و نسبی را تمرین کنید، ls را با گزینه‌های مختلف اجرا کنید و find را برای جستجوی دقیق استفاده کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. با pwd مسیر فعلی را ببینید و تغییرات مسیر را با cd و .. و ~ امتحان کنید.

۲. درخت `/etc/ssh` را با `tree` و سپس با `ls -R` ببینید؛ تفاوت را نوشتید.

۳. فایل‌های مخفی خانه‌ی خود را ببینید و اندازه‌ی آن‌ها را به صورت خوانا چاپ کنید.

۴. تفاوت `ls /etc` و `ls -d /etc` و `ls -ld /etc` را توضیح دهید.

۵. با find تمام فایل‌های خالی (اندازه‌ی صفر) دایرکتوری `/var` را پیدا کنید.

۶. تمام فایل‌های `.log` در `/var/log` که در آخر ۳ روز تغییر کردند را پیدا کنید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ pwd; cd /var/log; pwd; cd ../..; pwd; cd ~
$ tree -L 1 /etc/ssh
$ ls -R /etc/ssh | head -5
$ ls -lah ~/ | grep '^\.' | head -2
$ ls /etc | head -2
/etc
$ ls -d /etc
/etc
$ ls -ld /etc
drwxr-xr-x 1 root root 4096 Oct 4 08:15 /etc
$ find /var -type f -empty -o -type f -size 0 | head
$ find /var/log -type f -name '*.log' -mtime -3
```

✓ **بررسی نتیجه (Verify)**

`ls /etc` فهرست محتویات دایرکتوری را نشان می‌دهد. `ls -d /etc` فقط نام `/etc` را (بدون داخل) و `ls -ld /etc` مشخصات `/etc` خود را چاپ می‌کند.

Objective 2.4

وزن در آزمون: ۲

ساخت، کپی، جابه‌جایی و حذف

Creating, Moving and Deleting Files

فرمان‌های مدیریت فایل قدرتمند و بیرحمانه. `rm` سطل زباله ندارد؛ حذف‌شده برای‌ابد است (مگر اینکه دستگاه را خاموش کنید تا بازیابی دستی). قبل از حذف، پوشه‌ی فعلی و پوشه‌ی مقصد را با `pwd` و `ls` بررسی کنید.

mkdir

ساخت دایرکتوری؛ `-p` والدهای الزم را هم می‌سازد

```
$ mkdir -p ~/projects/web/backend
```

touch

ساخت فایل خالی یا به‌روزرسانی زمان دسترسی

```
$ touch notes.txt
```

cp

کپی فایل یا دایرکتوری (با `-r` ، بازگشتی)

```
$ cp -r src/ backup/
```

mv

جابه‌جایی یا تغییر نام

```
$ mv old.txt new.txt
```

rm

حذف فایل (هشدار!)

```
$ rm -i old.txt
```

rmdir

حذف دایرکتوری خالی

```
$ rmdir empty_dir
```

سوییچ‌های مهم

سوییچ	شکل بلند	کاربرد	مثال
`-i`	`--interactive`	پرسش قبل از حذف یا‌ونویسی (بسیار پیشنهاد می‌شود)	`rm -i file`
`-f`	`--force`	بدون پرسش (خطرناک!)	`rm -f file`
`-r`	`--recursive`	بازگشتی (برای دایرکتوری‌ها و محتویات)	`cp -r src dest`
`-v`	`--verbose`	نمایش هر عملیات انجام‌شده	`cp -v file1 file2`

```
student@ubuntu-lab: ~
$ mkdir -p ~/lab/{docs,images,scripts}
$ tree ~/lab
/home/student/lab
├── docs
├── images
└── scripts
$ touch ~/lab/docs/readme.md
$ cp -v ~/lab/docs/readme.md ~/lab/docs/readme.bak.md
$ mv ~/lab/docs/readme.bak.md ~/lab/docs/readme.backup.md
$ rm -i ~/lab/docs/readme.backup.md
rm: remove regular empty file '/home/student/lab/docs/readme.backup.md'? y
```

هشدار

هیچ‌گاه از `rm -rf` تا زمانی که کاملاً مطمئن نیستید استفاده نکنید. این فرمان بدون پرسش دایرکتوری و **تمام محتویات** را حذف می‌کند. شاخی کافی است یا علامت `/` جای عددی تایپ شود و تمام سیستم خطر میفتد.

نکته

`cp -r` و `mv` برای دایرکتوری‌های پلاک کار می‌کند، اما برای **جابه‌جایی درون دیسک** (same filesystem) `mv` بسیار سریع‌تر است.

Globbing and Wildcards

Globbing: الگوهای Wildcard

شِل قبل از اجرای فرمان، **الگوهایی شامل نویسه‌های ویژه** را **گسترش** (expansion) می‌دهد. این کار توسط شِل، نه خود فرمان، صورت می‌گیرد. الگوهای رایج:

الگو	معنی	مثال
`*`	صفر یا بیشتر نویسه‌ی هر چیزی	`*.txt` (همه‌ی فایل‌های متنی)
`?`	دقیقاً یک نویسه‌ی هر چیزی	`file?.txt` (file1.txt، file2.txt)
`[abc]`	هر یک از نویسه‌های داخل براکت	`[rR]eport` (report یا Report)
`[a-z]`	هر نویسه در بازه‌ی (مثلاً a تا z)	`page[1-5].pdf`
`[!abc]`	هر نویسه‌ی **غیر** daخل براکت	`file[!0-9].txt`
`{a,b,c}`	انشعاب؛ تمام ترکیبات	`file.{txt,md,pdf}`

student@ubuntu-lab: ~

```
$ ls *.txt  ## all txt files
$ rm -i report[0-9].pdf  ## report0 to report9
$ mkdir {jan,feb,mar}_{draft,final}  ## 6 directories
$ cp file.{old,new}  ## copies file.old to file.new
```

هشدار

Globbing در **غلاف** انجام می‌شود. اگر فایلی پیدا نشود، Bash خود الگو را آرگومان می‌فرستد. اگر در اسکریپت نقل‌قول استفاده کنید، Globbing غیرفعال می‌شود.

LAB 08

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

مدیریت فایل و Globbing

ساختاری پیچیده را بسازید، فایل‌ها را جابه‌جا کنید، نام‌ها را تغییر دهید و Globbing را استفاده کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱ ساختار `~/work/{2024,2025}/{jan,feb}/{draft,final}` را بسازید.

۲ در هر دایرکتوری `draft` سه فایل `report1.txt`، `report2.txt` و `report3.txt` بسازید.

۳ فایل‌های report را با Globbing به دایرکتوری‌های `final` کپی کنید.

۴ فایل‌های در `2024/jan/final` با نام `report` را با جستجوی Globbing حذف کنید.

۵ دایرکتوری‌های خالی در بازگشتی حذف کنید.

۶ همه‌ی تغییرات را با tree یا find نشان دهید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ mkdir -p ~/work/{2024,2025}/{jan,feb}/{draft,final}
$ touch ~/work/{2024,2025}/{jan,feb}/draft/report{1,2,3}.txt
$ cp ~/work/2024/jan/draft/*.txt ~/work/2024/jan/final/
$ rm -i ~/work/2024/jan/final/report*.txt
$ tree ~/work 2>/dev/null || find ~/work -type f
```

✓ **بررسی نتیجه (Verify)**

ساختار باید ۸ دایرکتوری draft و 8 دایرکتوری final داشته باشد. اگر tree نصب نیست، find با `-type f` همه فایل‌ها را نشان می‌دهد.

Objective 2.5

وزن در آزمون: ۲

خلاصه فصل دو و تمرین پیشرفته

Advanced File Operations Review

این فصل مهم‌ترین مهارت‌های کار با ترمینال را پوشش داده: راهنمایی، مسیریابی، فهرست‌گرفتن و جستجو. بقیه‌ی قدرت خط فرمان در فصل سوم (Pipe، grep، ویرایشگر و اسکریپت) است.

نکته‌ی آزمون

بیشترین سؤال‌های آزمون درباره‌ی **find** و **ls** است. بدانید `ls -la` شامل مالک و مجوز است، `find` بازگشتی جستجو می‌کند و `locate` سریع اما روی دیتابیس مبتنی است.

جلد دوم: قدرت پیشرفته

Volume 2: Advanced Power User

این جلد ادامه‌ی فصل دو است و تا فصل پنج ادامه می‌یابد. دروس و لابراتوارهای پایانی که شما را از **کاربر مقدماتی** به **مدیر سیستم تازه‌کار** تبدیل می‌کند.

فصل ۳: قدرت خط فرمان (۳۰ صفحه)

۳٫۱ Redirect و Pipe: جریان‌های stdin/stdout/stderr؛ علامت‌های `>`، `>>`، `|`، `tee`. **۳٫۲ Grep و Regex:** جستجوی الگو، عبارات منظم، `grep -i`، `grep -v`. **۳٫۳ آرشیو:** tar، gzip، xz، zip برای backup. **۳٫۴ پردازش متن:** cut، sort، uniq، wc، sed برای تغییر متن. **۳٫۵ ویرایشگرها:** nano و vim؛ جابه‌جایی، جستجو، جایگزینی. **۳٫۶ اسکریپت Bash:** شروع scripting با متغیرها و حلقه‌ها.

فصل ۴: سیستم‌عامل (۳۰ صفحه)

۴٫۱ سخت‌افزار: lscpu، lsblk، free برای مشاهده‌ی سیستم. **۴٫۲ فایل‌سیستم:** ext4، mount/unmount. **۴٫۳ FHS:** درخت استاندارد `/etc`، `/var`، `/usr`، `/home`. **۴٫۴ فرایند و سرویس:** ps، top، systemctl، journalctl. **۴٫۵ شبکه:** مفاهیم IPv4/IPv6، ip addr، ss، ping، dig. **۴٫۶ Netplan و networkd:** تنظیم IP ثابت.

فصل ۵: امنیت و مجوزها (۴۰ صفحه)

۵٫۱ کاربران: UID/GID، /etc/passwd، /etc/shadow، sudo. **۵٫۲ ایجاد کاربر:** useradd، adduser، groupadd. **۵٫۳ مجوزها:** chmod (عددی و نمادی)، chown، umask. **۵٫۴ مجوزهای ویژه:** setuid، setgid، sticky bit. **۵٫۵ لینک‌ها:** نمادی و hard link. **۵٫۶ فایل‌های ویژه:** /etc/sudoers، /etc/fstab.

پروژه‌ی پایانی و آزمون (۱۰ صفحه)

اسکریپت صحی سیستم (healthcheck) و سؤالات تمرینی آزمون.

هدف: **۱۹۰ صفحه کل**؛ جلد ۱ و جلد ۲ هر کدام ۹۰–۱۰۰ صفحه.

جلد اول

جلد اول: پایه‌ای که همه‌چیز روی آن ساخته می‌شود

این جلد برای کسی نوشته شده که لینوکس را فقط نصب کرده و حالا روبه‌روی یک ترمینال سیاه نشسته است. از ساخت ماشین مجازی روی VMware Workstation شروع می‌کنیم، با تاریخچه و فلسفه‌ی متن‌باز آشنا می‌شویم و بعد قدم‌به‌قدم خط فرمان را یاد می‌گیریم: هر فرمان با ساختار کلی، جدول سوییچ‌های رایج، مثال‌های واقعی، هشدارها و لابراتوار سناریومحور. مطالب از ساده به سخت چیده شده تا بتوانید در خلوت خودتان هم پیش بروید.

در این جلد

- آماده‌سازی آزمایشگاه VMware و نصب Ubuntu
- تاریخچه، توزیع‌ها، apt و dpkg و snap، مجوزها، SSH
- Bash، متغیر، راهنما، ls و cd، cp و mv و rm، find
- cat و Redirect، less، grep و Pipe، tar و Regex

پس از این جلد می‌توانید

- ✓ با اطمینان در درخت فایل‌ها حرکت کنید
- ✓ فایل و پوشه بسازید، کپی و منتقل و حذف کنید
- ✓ هر فایلی را با find و locate پیدا کنید
- ✓ متن را با grep و Pipe فیلتر کنید
- ✓ بسته نصب و حذف کنید و نسخه پشتیبان بگیرید

```
student@ubuntu-lab: ~

student@ubuntu-lab:~$ sudo apt install confidence
Reading package lists... Done
Setting up confidence (24.04-LTS) ...
student@ubuntu-lab:~$ echo "Volume 1 complete"
Volume 1 complete
student@ubuntu-lab:~$
```

NEXT · VOLUME 02

ویرایشگر، اسکریپت، شبکه و امنیت

Bash scripting، vim، فرایندها، netplan و networkd، کاربران و chmod

مدرس و طراح لابراتوار

محمدرضا بیدختی

MOHAMMAD REZA BEIDOKHTI

UBUNTU 24.04 LTS

VMware Workstation 17+

LPI Linux Essentials 1.6